队伍编号	MCB2302973
赛道	B

基于计算机视觉的坑洼道路检测和识别

摘 要

坑洼道路检测和识别是地质勘探、航天科学和自然灾害等领域一项重要研究，快速精准识别道路坑洼状态对分析和模拟道路形态具有重要意义。本文针对坑洼道路检测和识别问题，基于改进**超分辨率生成对抗网络**和改进 **YOLOv8** 检测网络建立识别模型，依据训练收敛性和检测精度等维度对识别模型进行评估。

针对问题一，首先选取超分辨率生成对抗网络获得虚拟特征以解决训练样本数量少的问题，加入空间和通道**注意力机制**引导生成器特征生成接近真实特征的道路坑洼特征图，使用图像分块对待训练的图像进行分块和重组，在提升图像占比的同时保留中心目标点。然后选取 YOLOv8 检测网络作为目标检测的基础框架，利用**深度卷积神经网络**改进检测网络中的卷积核，引入双路径注意机制以提升检测网络对目标的关注度，采用**共线方程**和**高斯滤波**提取坑洼道路边缘特征，利用**特征增强**模块连接多通道特征值以生成待检测的特征图，进而增强网络的特征提取能力，再将得到的特征图输入值道路检测网络实现特征提取与目标检测，针对检测过程中目标丢失的问题，在已有 IoU 损失函数的基础上采用 **CIoU** 调整检测过程中存在的损失函数，进一步提升坑洼道路的检测精度。通过级联改进超分辨率生成对抗网络和改进 YOLOv8 网络最终建立坑洼道路检测和识别模型，级联后的算法将达到快速、高精度目标检测与识别。

针对问题二，基于问题一所建立的模型进行参数训练，分别选取 SGD 和 Adam 优化器对模型参数进行分组优化，得到损失小和精度高的最佳权重，基于最佳权重值以实现模型评估和验证。本文从模型**收敛性**和**检测精度**两个维度对模型进行评估，选取训练损失、准确率、精确率、召回率、F1 分数和 mAp 等指标评估模型性能，对设计的检测模型进行消融实验和横向对比实验。实验结果表明，经 SGD 优化的检测模型相较于 Adam 优化后的模型可以达到更快的收敛，且精度较高。加入注意力机制和深度卷积核可明显提升模型性能，在消融实验中改进后的 YOLOv8 模型与基准 YOLOv8 相比，精度提升了 10.9%，$mAp_{.5}$ 提升了 14.2%，$mAp_{.5:.95}$ 提升了 12.2%，在横向对比实验中改进后的 YOLOv8 准确率高于其他检测模型，相较于基准 YOLOv8 精度提升了 12.23%，验证了所建立模型的有效性，且通过模型剪枝的方式降低了器参数量，实现了模型轻量化，因此基于改进超分辨率生成对抗网络和改进 YOLOv8 的坑洼道路检测模型可以达到较高精度的目标识别。

针对问题三，在问题一模型和问题二权重的基础上，利用测试数据进行模型验证，将测试图像输入至检测模型中得到识别结果，识别结果表明，4942 张测试图片中坑洼道路有 4045 张，正常道路有 897 张，在坑洼道路图片中，对坑洼位置识别置信度在 80% 以上的图像有 2597 张，仅有 68 张图片识别置信度小于 50%，但具有坑洼特征的测试图像均已识别，说明识别结果的有效性。检测结果表明，利用问题一建立的模型不仅可以精确识别单个坑洼目标，还可以识别多个坑洼目标，且具有对遮挡、重叠、远距离、模糊小目标的检测能力，表明本文坑洼道路模型的有效性与泛化性。

关键词：YOLOv8；超分辨率生成对抗网络；注意力机制；特征增强；特征生成

目录

一、问题重述 1
1.1 问题背景 1
1.2 任务目标 1
二、问题分析 1
三、模型假设 2
四、符号说明 2
五、数据预处理 3
六、问题一模型建立与求解 4
6.1 问题一分析 4
6.2 基于改进超分辨率生成对抗网络的特征生成模型 4
6.2.1 SRGAN 网络结构分析 5
6.2.2 基于 CBAM 的生成网络结构分析 5
6.2.3 基于图像分块的特征生成结构分析 6
6.3 基于改进的 YOLOv8 坑洼道路检测识别模型 7
6.3.1 改进的深度卷积神经网络 8
6.3.2 特征增强模块（FEM） 10
6.3.3 基于共线方程和高斯滤波的坑洼道路边缘检测 11
6.3.4 道路检测网络特征提取 13
6.3.5 基于改进 IoU 的目标检测 14
6.4 YOLO 模型剪枝 15
6.5 模型测试 16
七、问题二模型建立与求解 16
7.1 训练参数 16
7.2 数据集简介 17
7.3 模型评价指标 17
7.4 模型性能评估 18
7.4.1 收敛性分析 18
7.4.2 分类精度分析 19
7.5 消融实验 20
7.6 模型对比实验 20
7.7 测试结果展示 21
八、问题三模型建立与求解 24
8.1 基于改进 YOLOv8 模型的识别结果 24
8.2 基于改进 YOLOv8 模型的检测结果 25
九、模型评价 27
9.1 模型优点 27
9.1.1 问题一优点 27
9.1.2 问题二优点 27
9.1.3 问题三优点 27
9.2 模型缺点 27
参考文献 28
附录 29

一、问题重述

1.1 问题背景

坑洼道路检测和识别作为一种计算机视觉任务，可以通过数字图像（通常是地表坑洼图像）识别出存在坑洼的道路。这对于地质勘探、航天科学和自然灾害等领域的研究与应用具有重要意义，可以帮助在地球轨道上识别坑洼，以及分析和模拟地球表面的形态。但在坑洼道路检测任务中，图像特征是复杂多变的，传统的分类算法往往不能取得较好的效果，而深度学习技术的发展为坑洼道路检测提供了新的解决方案。

深度学习因其具有很强的特征提取和表示能力，可以从图像中自动提取出重要特征，利用深度学习可以在坑洼图像分类任务中提取到坑洼的轮廓、纹理和形态等特征，并将其转换为更为容易分类的表示形式，同时可通过迁移学习和知识蒸馏等技术进一步提升分类性能。

1.2 任务目标

基于计算机是绝对阿坑洼道路检测和识别需要通过对已标记的道路图像进行分析、特征提取与建模，进而自动识别新道路图像的坑洼状态。

问题一：结合附件图像文件，提取图像特征，建立识别率高、速度快、分类准确的用于识别图像中道路状态的模型。

问题二：对问题一中构建的模型进行训练，并从不同维度进行模型评估。

问题三：利用已训练的模型识别测试集中的坑洼图像，并保存识别结果。

二、问题分析

针对问题一，需要结合附件中的图像文件，提取图像特征并建立识别率高、快速、分类准确的模型，通过对附件数据进行量化分析，拟选取改进超分辨率生成对抗网络进行特征生成以扩充特征图，在超分辨率生成对抗网络的基础上引入通道和空间注意力机制以实现对目标特征的精准提取，利用生成对抗网络结构生成接近真实数据的坑洞特征。拟选取改进的 YOLOv8 检测网络实现道路检测识别，引入双路径注意机制和特征增强模块提升特征分解精度，将共线方程与高斯滤波结合实现对真实图片的坑洼边缘特征提取，然后利用检测网络对特征图进行识别以完成目标检测，引入 CIoU 提升最终的检测精度，进而建立了基于计算机视觉的坑洼道路检测和识别模型。

针对问题二，需基于问题一所建立的模型进行训练，并从不同维度进行模型评价，拟分别选取 SGD 和 Adam 优化器对模型参数进行分组优化，进而获得最佳权重值以实现模型评估和验证。拟选取训练损失、准确率、精确率、召回率、F1 分数和 mAp 等指标评估模型性能，从模型收敛性和检测精度两个维度进行评估，并对设计的检测模型进行消融实验和横向对比实验。因此基于改进超分辨率生成对抗网络和改进 YOLOv8 的坑洼道路检测模型可以达到较高精度的目标识别。

针对问题三，在问题一模型和问题二权重的基础上，利用测试数据进行模型验证，将测试图像输入至检测模型中得到识别结果，进而验证本文坑洼道路模型的有效性。

针对题目有待解决的问题，本文列出如图 1 所示的问题分析框架。

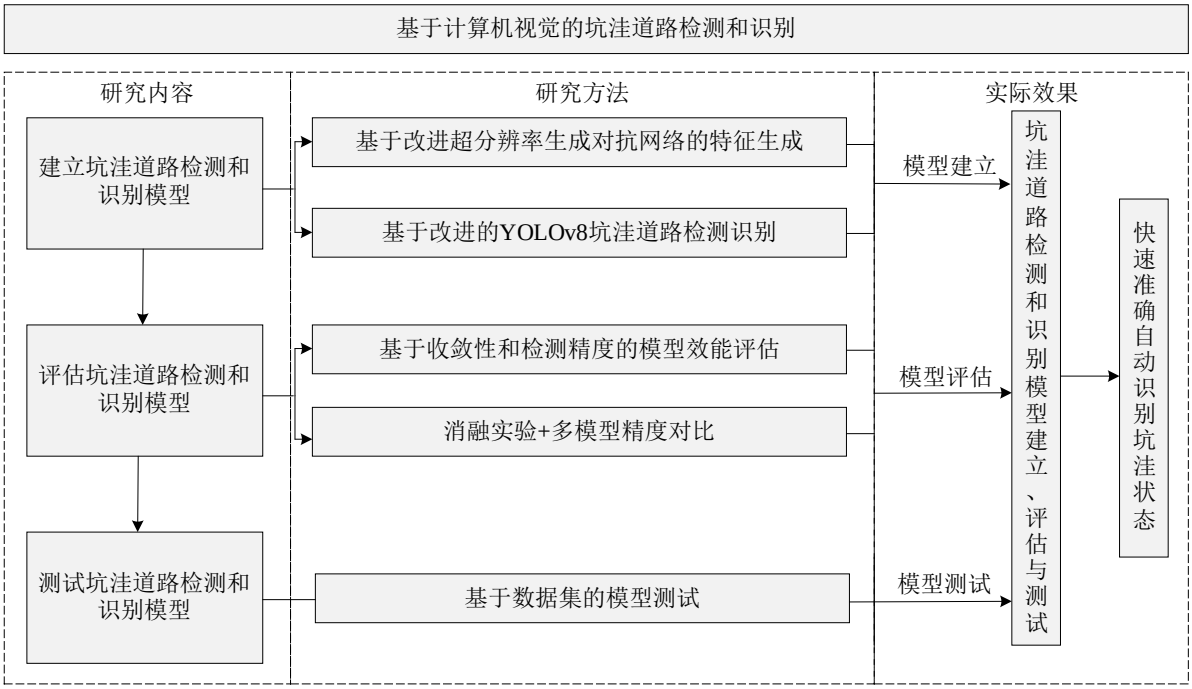


图 1 问题分析框架图

三、模型假设

1、假设道路坑洼状态仅包括路面坑洼出现裂缝，不包括井盖下凹等情况
2、假设训练数据与测试数据仅有少量重复图片
3、假设训练与测试数据真实可靠

四、符号说明

符号	含义
L_G	生成对抗网络损失
M_c	通道注意力
M_s	空间注意力
(X_p, Y_p, Z_p)	坑洼图片像素点
$C(x)$	归一化函数
L_{IoU}	CIoU
Acc	准确率
Pre	精确率
Rec	召回率
Fsc	F1 分数
mAp	平均精度

注：未列出符号及重复的符号以出现处为准

五、数据预处理

为使建立的模型能够满足坑洼道路检测要求，在对问题进行正式分析前，需要对数据进行分析，并通过对应的工具进行数据预处理以保证模型验证准确性。附件中图片数据有 normal 和 pothole 两类图像，表 1 显示了附件图片可用数量。

表 1 附件道路图像数据

	Normal	Pothole
图像数量	266	35

附件中所给出的正常道路图像如图 2 所示，图 2 列举部分图像。



图 2 正常道路图像

附件中所给出的正常道路图像如图 3 所示，图 3 列举部分图像。

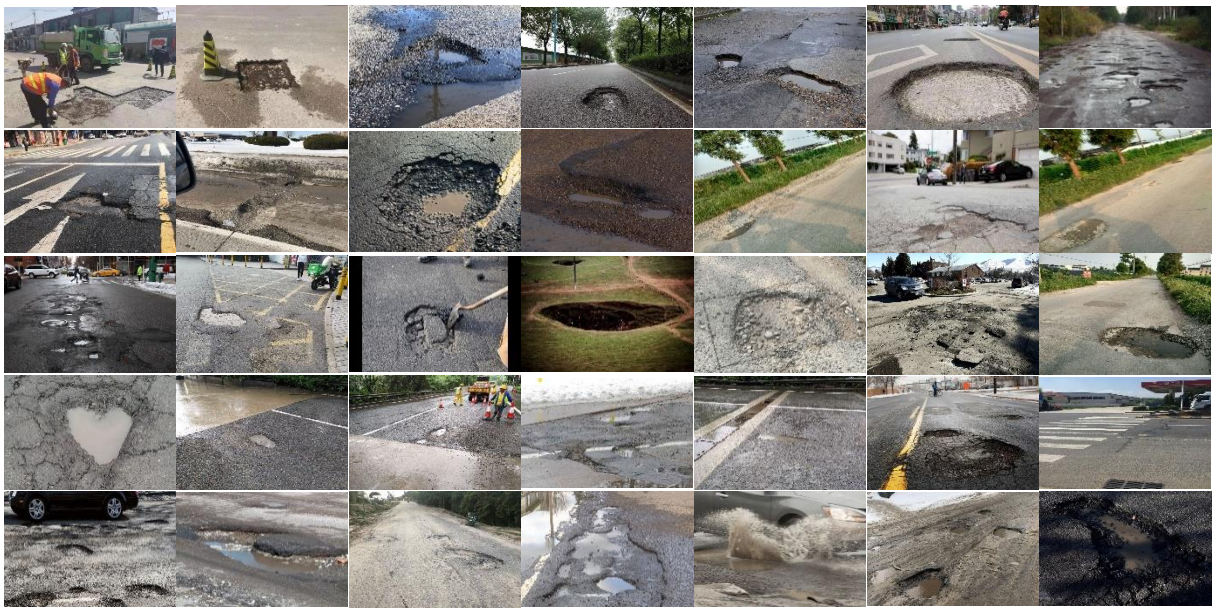


图 3 坑洼道路图像

针对具有坑洼特征的图片，使用 XML 文件记录坑洼位置的标签信息，利用 Labelimg 工具标记图像中的坑洼位置为 Pascal VOC 数据，以图 4 为例，在同一图像中对坑洼目标进行标记，如图 4 所示，然后将 VOC 标签转化为 YOLO 标签。



图 4 YOLO 坑洼图像标签

六、问题一模型建立与求解

6.1 问题一分析

基于深度学习的图像分类算法以卷积神经网络作为图像特征提取网络，基于预先设定的卷积核和图像特征矩阵进行卷积计算，进而对图像目标进行特征提取，最终可以得到不同维度的纹理、色彩和几何等图像特征。

基于视觉对道路坑洼进行分类，在各种照明场景下，物体在形状、比例和阴影方面往往会存在较大的差异，通常带有高度复杂的背景（如存在遮挡、目标不清晰等）会对图像分类造成干扰，因此需要具有鉴别细节、模糊、隐藏特征能力的模型对各种场景下的坑洼道路进行识别。

针对道路坑洼图像特征复杂且环境多变的难点，本文建立了基于改进 YOLOv8 和超分辨率生成对抗网络级联的坑洼道路识别模型。模型首先通过，并结合共线方程和高斯滤波边缘检测细化坑洼目标的边缘粒度，进而在 YOLOv8 的基础架构上得到改进的 YOLOv8 检测算法。其次针对坑洼目标特征不足的问题，引入改进的超分辨率生成对抗网络对目标进行特征生成，并引入 CBAM 机制提升生成网络对坑洼目标的关注度，使用重叠分块对图像进行分块与重组，将特征生成网络与检测网络进行级联以达到高识别率、快速、分类准确的目的。针对问题一有待解决的问题，本文绘制了如图 5 所示的问题分析框架图。

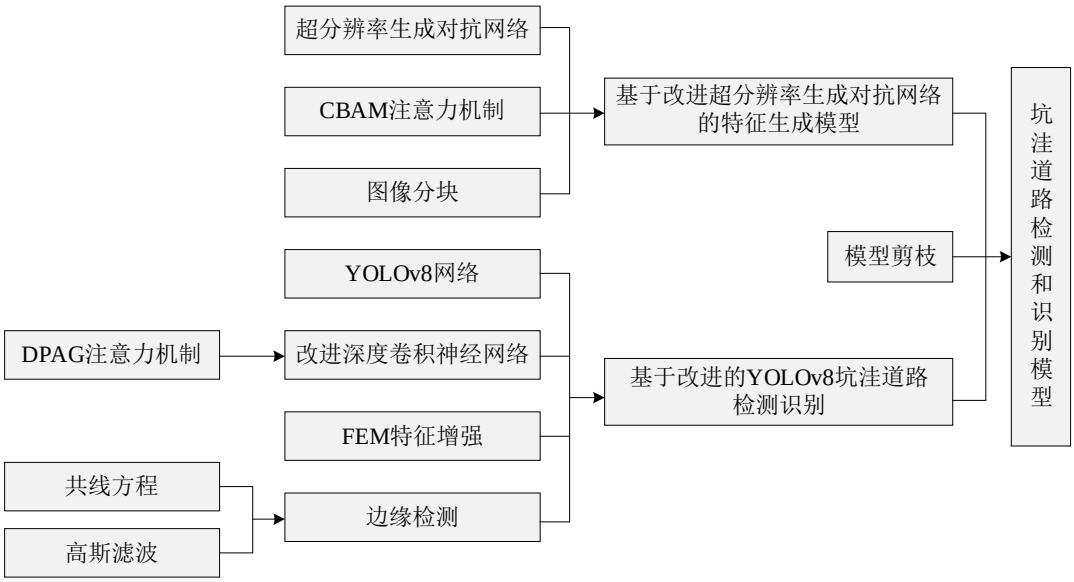


图 5 问题一分析框架图

6.2 基于改进超分辨率生成对抗网络的特征生成模型

针对数据集较少的问题，本文通过引入虚拟特征来增强目标特征数据集，基于增强型超分辨率生成对抗网络架构，加入空间注意力对图像进行超分辨率处理，以达到补充图像特征集的目的，同时提高模糊图像在训练过程中的分辨率。

生成对抗网络由生成网络和判别网络组成，通过互相学习和博弈的方法实时调整判别网络参数，最终使得判别网络无法判断生成网络的输出结果的真实性，本模型则通过引入超分辨率生成对抗网络（Super-Resolution Generative Adversarial Networks, SRGAN）生成目标特征[1]，在原有特征数据集的基础上提取更多符合实际坑洼道路的特征。

6.2.1 SRGAN 网络结构分析

其中，SRResNet 作为超分辨率生成对抗网络的基本框架，通过密集残差模块组成对抗网络的基本单元，每个连接层会以历史输入层的结果作为额外输入结果以达到稠密连接。

基于稠密连接的方式可以增加模块特征间的信息传递，每层可直接利用损失函数的梯度值以避免梯度消失过快和过拟合，进而可以构建深度训练网络。

对抗网络一般通过判别器判断输入图片是否真实，以“0”和“1”进行标记，而 SRGAN 则通过平均判别器计算出生成图像接近真实图像的概率，最终让标记概率不断接近“0”或“1”。计算公式如式(1)所示。

$$\begin{cases} D\left(x_r, x_f\right) = \sigma(\mathrm{C(Real)}) - E[\mathrm{C(Fake)}] \to 1 \\ D\left(x_f, x_r\right) = \sigma(\mathrm{C(Fake)}) - E[\mathrm{C(Real)}] \to 0 \end{cases} \tag{1}$$

其中 $D(x) = \sigma(C(x))$ 为一般标准判别器，σ 为 *sigmoid* 函数，$C(x)$ 为非变换判别器输出，$E(x)$ 为对所有假数据取平均值。此时，判别器 D 的损失函数如式(2)所示。

$$L_D^{Ra} = -E_{x_r}\left[\log\left(1 - D_{Ra}\left(x_r, x_f\right)\right)\right] - E_{x_f}\left[\log\left(D_{Ra}\left(x_f, x_r\right)\right)\right] \tag{2}$$

生成器 G 的损失函数如式(3)所示。

$$L_D^{Ra} = -E_{x_r}\left[\log\left(D_{Ra}\left(x_r, x_f\right)\right)\right] - E_{x_f}\left[\log\left(1 - D_{Ra}\left(x_f, x_r\right)\right)\right] \tag{3}$$

上式中 $x_f = G(x_i)$ 和 x_i 为低分辨率输入图像，生成器 G 的损失函数包含了 x_f 、x_r，增加了真实图像在训练过程中的影响，对于小目标图像，这种改进更有利于判别器学习更真实的边缘等细节特征。

感知层损失函数用于衡量生成图像与真实图像间的像素损失，感知损失如式(4)所示。

$$L_1 = E_{x_i}\left\|G\left(x_i - y\right)\right\|_1 \tag{4}$$

基于感知层损失可以得到生成器完整的损失函数，如式(5)所示。

$$L_G = L_{\mathrm{percep}} + \lambda L_G^{Ra} + \eta L_1 \tag{5}$$

其中 λ 和 η 为平衡各损失项的平衡系数。

SRGAN 在进行训练时可以有效避免部分特征点稀疏而导致弱监督的问题，且采用激活前的特征不会改变图像整体质量，适合坑洼道路图像的数据增强，本文以 SRGAN 作为主干网络将得到更为逼真的特征值。

6.2.2 基于 CBAM 的生成网络结构分析

由于检测目标在画面中的位置与环境间存在耦合的关系，如图 6 所示，施工人员、铲子、汽车与道路坑洼位置存在遮挡，在识别过程中会存在误判或模糊情况而导致精度下降，因此在 SRGAN 网络中引入注意力机制，在道路环境中添加坑洼目标的位置依赖，通过生成网络训练坑洼目标的空间信息权重，进而提高网络对特征的提取能力，使得生成网络得出更加精细的边缘特征。



（a）potholes00000　　（b）potholes9　　（c）potholes31

图 6 存在遮挡或误判坑洼道路场景

CBAM 模块兼顾通道注意力和空间注意力，通过全局最大池化和全局平均池化方式保证信息最大化保留[2]，在注重通道信息完整的同时还注重空间信息，CBAM 网络结构如图 7 所示，由通道和空间模块组成。

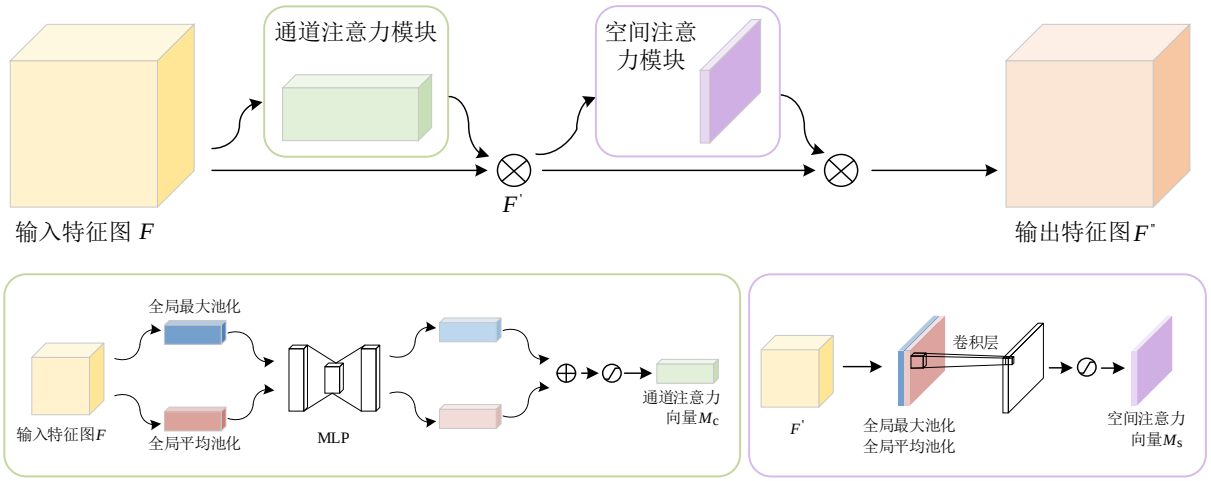


图 7 CBAM 模块

1）通道注意力分别采用最大池化操作和平均池化聚合输入特征图的空间信息，最终得到两个多维池化特征图，输出为两个 C 维池化特征图，分别为 F_{avg} 和 $F_{\max}$。然后将 F_{avg} 和 $F_{\max}$ 输入多层感知器 MLP，得到两个参数为 $1\times1\times C$ 的通道注意力图。为减少参数量，隐层神经元需要压缩输入信息，神经元个数为 C/r，r 被称作压缩比。将上一步的输出进行相加，便可得到通道注意力 M_c 如式(6)所示。

$$\begin{aligned}\mathbf{M}_c(\mathbf{F}) &= \sigma(\mathrm{MLP}(\mathrm{AvgPool}(\mathbf{F})) + \mathrm{MLP}(\mathrm{MaxPool}(\mathbf{F}))) \\ &= \sigma\left(\mathbf{W}_1\left(\mathbf{W}_0\left(\mathbf{F}_{\mathrm{avg}}^{\mathrm{c}}\right)\right) + \mathbf{W}_1\left(\mathbf{W}_0\left(\mathbf{F}_{\max}^{\mathrm{c}}\right)\right)\right)\end{aligned} \tag{6}$$

2）空间注意力对于 F' 最先按照通道维度进行平均池化操作和最大池化操作，输出两个特征图 F_{avg} 和 $F_{\max}$，输出的参数为 $1\times H\times W$，并在通道维度上拼接输出的两个特征图，即可拼接后的空间特征图。对于拼接后的特征图，使用尺寸为 7×7 大小的卷积层进一步生成空间注意力 M_s 如式(7)所示。

$$\begin{aligned}\mathbf{M}_s(\mathbf{F}) &= \sigma\left(f^{7\times7}([f_{AP}(\mathbf{F}); f_{mP}(\mathbf{F})])\right) \\ &= \sigma\left(f^{7\times7}\left(\left[\mathbf{F}_{\mathrm{avg}}^{\mathrm{s}}; \mathbf{F}_{\max}^{\mathrm{s}}\right]\right)\right)\end{aligned} \tag{7}$$

式中 f_{AP} 表示平均池化，f_{mP} 表示最大池化，引入注意力机制将有助于生成网络在训练时得出较多的坑洼道路纹理和边缘信息，提高了模型的性能与准确率。更多测试图片与精度对比分析将在问题二中给出。

6.2.3 基于图像分块的特征生成结构分析

坑洼图像在经过超分辨生成网络处理后，图像分辨率将会得到提升，但由于检测目标的画面大小不变，经过变换后的图像放入检测网络中无法充分利用由生成网络得到的信息，因此在超分辨生产网络的基础上引入图像分块将超分辨率后的图像按照比例划分为若干可检测的图，再将其输入至检测网络中，如图 8 所示。

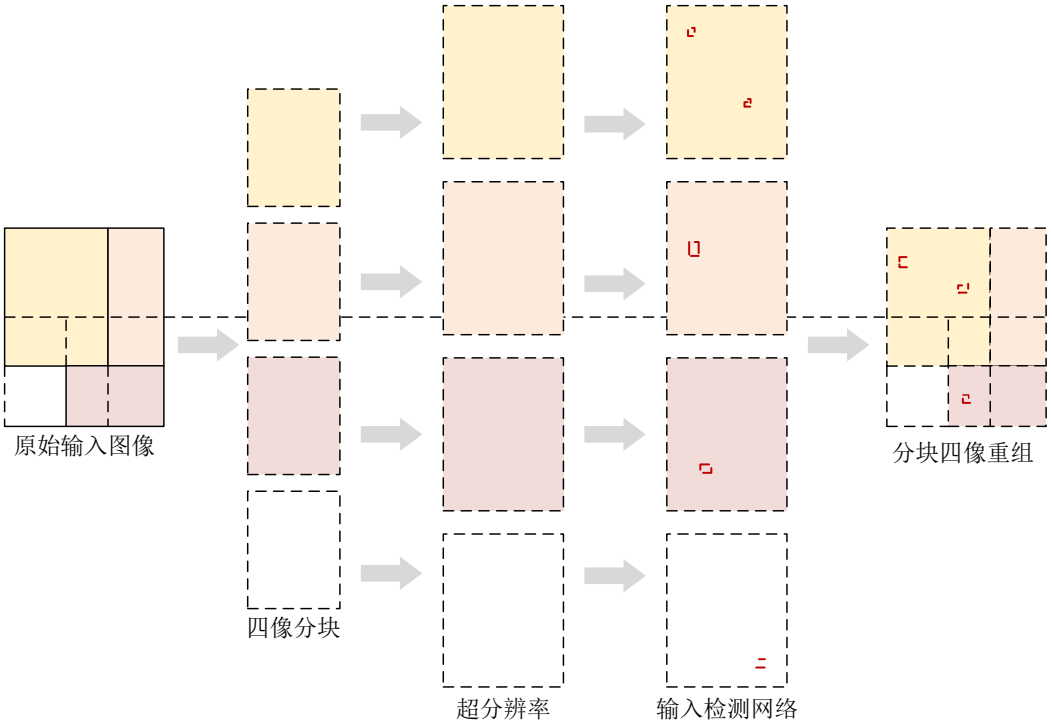


图 8 图像分块划分过程

本文所输入的目标和生成网络得到的特征值将会基于图像分块进行目标分割，以获取目标中心点，如图 9 所示。



图 9 图像分块结果

如图 9 所示，原坑洼目标在图片中的占比为 20.184%，分块后的在每个 block 中占比为 81.635%，提升近 4 倍，且本文方法会首先定位到目标点，再以目标点为中心进行非重叠分割，在保证图像占比提升的同时又不会丢失中心目标点。

6.3 基于改进的 YOLOv8 坑洼道路检测识别模型

在本文中，选择轻量级的 YOLOv8s 作为基础深度学习模型，具有较好的检测性能，可以检测图像中的小目标，应用于道路检测中可以完成坑洼目标的高精度检测与识别。

YOLOv8s 是从 YOLOv8 算法中衍生的轻量级参数结构，其包含骨干网络（Backbone）颈网络（Neck）和预测头（Head），骨干网络利用卷积运算从 RGB 彩色图像中提取各种尺度的特征[3]，颈网络合并骨干网络所提取的特征，头层则负责预测目标类别，并利用三组不同大小的检测器来选择和检测图像内容。其中基础 YOLOv8 网络结构如图 10 所示。

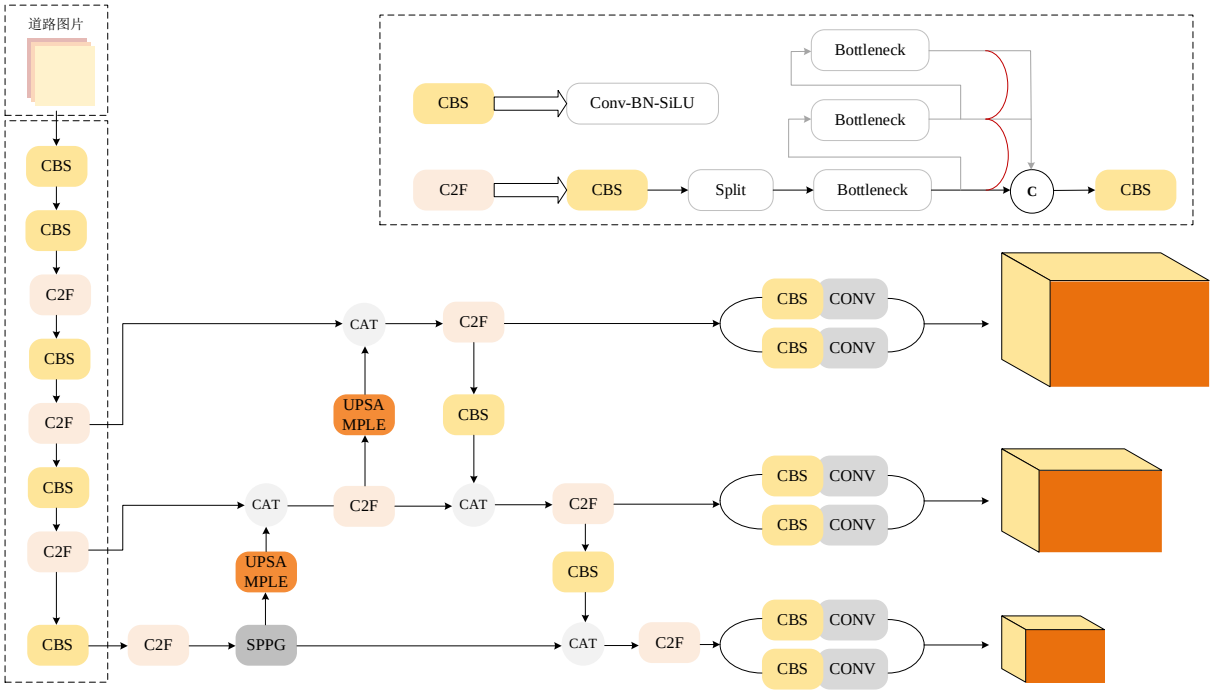


图 10 基础 YOLOv8 网络结构

具体实现步骤如下。

1）bbox 积分转换为 4d bbox

对 Head 输出的 bbox 分支进行转换，利用 Softmax 和 Conv 计算将积分形式转换为 4 维 bbox 格式。

2）维度变换

YOLOv8 输出特征图尺度为 80×80、40×40 和 20×20 的三个特征图，Head 输出分类和回归共 6 个组特征图。然后将 3 个不同尺度的类别预测分支、bbox 预测分支进行拼接形成三组输出结果，并进行维度变换。

3）解码还原到原图尺度

分类预测分支进行 Sigmoid 计算，而预测分支需要对其进行解码，还原为真实原图。

4）阈值过滤

遍历 batch 中的每张图，采用 score_thr 进行阈值过滤。

5）还原到原图尺度

基于前四步结果，将剩下的检测框还原到网络输出前的原图尺度。

6.3.1 改进的深度卷积神经网络

坑洼检测的目的在于确定给定路面图像中是否存在坑洼现象[4]，基于数据驱动模式可以采用改进的深度卷积神经网络（DCNN）改进原有的卷积网络结构，深度卷积神经网络使用原图像对应标记图像作为输入，深度卷积（DCNN）用于替代原始网络主干和颈部卷积来获得粗略的分数图，然后采用条件随机字段产生最终输出，DeepLabv3+会为图像中的每个像素分配一个语义标签 pothole。具体结构如图 11 所示。

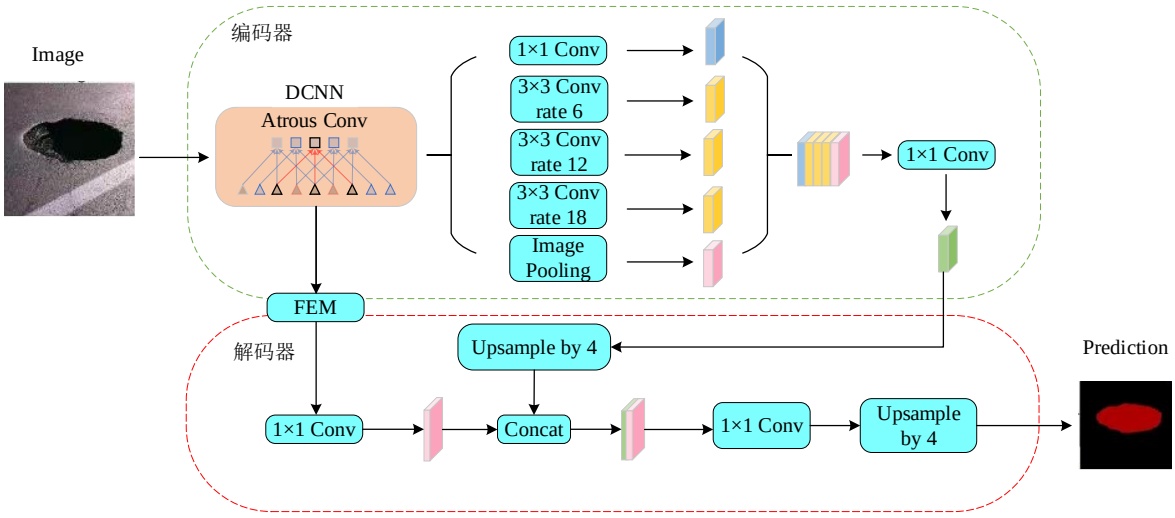


图 11 DCNN 结构图

传统卷积仅仅将卷积核应用于输入特征图的每个通道，并将每个通道的卷积结果相加，以创建单个输出特征图。在输入特征图的所有通道上重复该过程以创建多个输出特征图。相反，DeepLav3+由两个步骤组成。首先利用深度卷积产生具有与输入特征图相同数量通道的中间特征图，其中使用相应的卷积核独立地卷积输入特征图的每个通道。然后，在逐点卷积步骤中，将 1×1 卷积核应用于中间特征图的每个通道，得到最终输出特征图。DeepLav3+的主要优点是显著减少了参数数量和计算复杂性。与普通卷积相比，DeepLav3+可以在显著降低计算量的同时保持较好的精度。

除此之外，在 Concat 中添加了双路径注意机制（DPAG）[5]，平衡了检测速度和特征提取能力。DPAG 融合了 BCAM 和 CAM 注意力模块，前者实现双通道注意力，而后者学习图像位置信息，在通道层引入了一个附加路径，促进了信息提取。DPAG 结构如图 12 所示，BCAM 和 CAM 密切交互以提取通道和空间特征，其中 BCAM 通过通道关系门和位置关系门来增强通道相关性和特征精度，CAM 通过获取空间信息来定位实体。

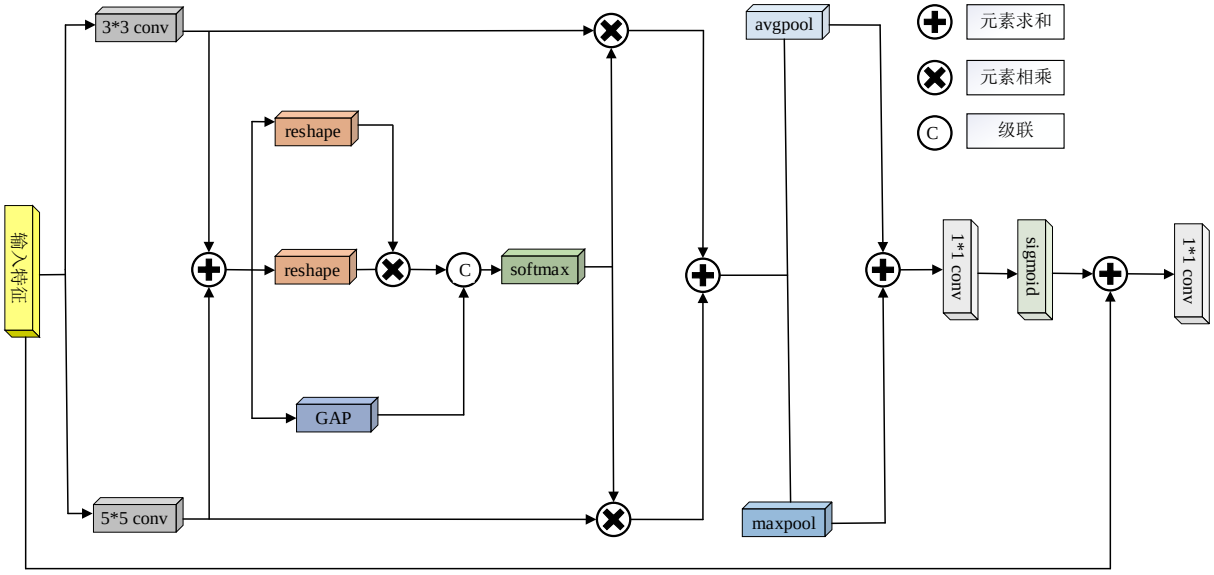


图 12　DPAG 结构图

通过 DPAG 的特征吸收和重构过程，像素获得根据权重值识别其重要性的单个权重。信道注意 BCAM 的运算如下：传输的特征图分别由标准卷积 Conv3 和 Conv5 处理，然后将两者合并以获得表示为“+”的卷积层。在全局平均池（GAP）之后，将重新整形的行和列信息相乘并级联以获得特征图，然后将其通过 softmax 层，以获得两组的学习权重，以获得通道注意力输出。

而空间注意力 CAM 基于通道注意力的输出，使用平均池（Avgpool）和最大池（Maxpool）。在 1×1 卷积和 Sigmoid 之后，最终的空间保持为 1×1 卷积形式。整个计算过程如式(8)、(9)、(10)、(11)、(12)所示。

$$\phi = Conv3(x) + RepConv5(x) \tag{8}$$

$$\delta = Softmax(Creshape(\phi) \times \phi; GAP(\phi))) \tag{9}$$

$$\Lambda = \delta \times Conv(x) + \delta \times RepConv(x) \tag{10}$$

$$\Omega = Avgpool(\Lambda) + Maxpool(\Lambda) \tag{11}$$

$$\Delta = Sigmoid(Conv(\Omega)) \tag{12}$$

其中，定义元素求和运算表示为“+”（元素求和），元素乘法运算表示为”×“（元素生成），通道串联表示为 C（级联）。

6.3.2 特征增强模块（FEM）

特征增强模块（Feature Enrichment Module, FEM）在神经网络中使用特征图来预测目标。由于特征图中包含的信息量对预测输出有直接影响，而主网络提取特征的能力有限。在网络的早期阶段，特征图处理不足，导致低层次特征图缺乏语义信息，导致检测小物体的适用性有限。采用 FEM 特征增强模块在多个分支上使用各种卷积和尺度构建多分支结构来连接多通道特征图，从而水平扩展网络宽度，FEM 结构如图 13 所示。

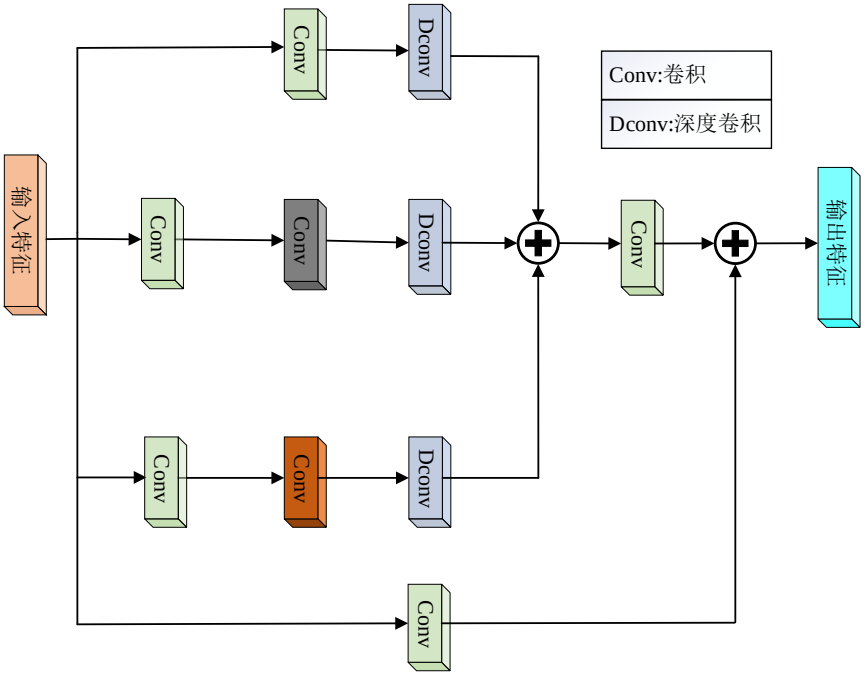


图 13 FEM 结构图

FEM 提高了网络在检测小尺寸物体时的适应性、灵敏度。为了增强网络的特征提取能力，在主网络中包括 FEM 模块，以提取与先前卷积层协同工作的全局特征，从而提高检测性能。此外，中间的两个分支结合了扩张的卷积层，以拓宽感受域并增加上下文信息，从而提高特征的有效性。图 13 显示了 FEM 的四个分支结构，前三个分支执行 1×1 卷积运算，以处理和调节特征图通道的数量，以便进一步处理。第四个分支包含一个残差结构，该结构在输出端生成等效映射，保留了对检测小目标有效的高质量特征。剩下的三个分支由级联的 3×3 传统卷积和扩张卷积组成，用不同尺度的卷积提取更细

的粒度目标特征。FEM 的计算过程如式(13)、(14)、(15)、(16)所示。

$$Y_1 = f_{\text{dconv1}}^{3\times3}\left| f_{\text{conv}}^{1\times1}(\text{X}) \right. \tag{13}$$

$$Y_2 = f_{\text{dconv3}}^{3\times3}\left\{ f_{\text{conv}}^{3\times3}\left[f_{\text{conv}}^{1\times1}(\text{X}) \right] \right\} \tag{14}$$

$$Y_3 = f_{\text{dconv5}}^{3\times3}\left\{ f_{\text{conv}}^{3\times3}\left[f_{\text{conv}}^{1\times1}(\text{X}) \right] \right\} \tag{15}$$

$$Z = \text{Concat}\left(Y_1, Y_2, Y_3\right) \oplus \text{X} \tag{16}$$

6.3.3 基于共线方程和高斯滤波的坑洼道路边缘检测

为了准确计算道路坑洼的形状，引入边缘检测算法识别坑洞部分，检测坑洞的闭合边缘，使用闭合路径可以精确检测坑洼位置的边缘，通过边缘算子提取的闭合边缘将视为路面与坑洞之间的边缘特征，当检测到的边缘不够清晰或没有闭合时，检测器将会检查每个凹坑的边缘，以确保识别精度，最终返回瞄框横纵坐标用于表示图像中的坑洞边缘，检测过程如图 14 所示。

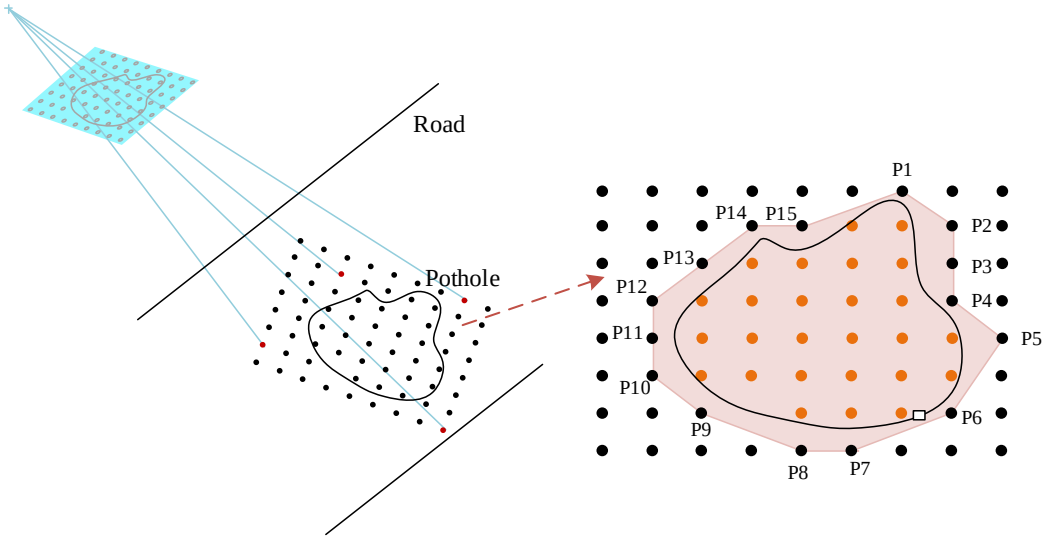


图 14 坑洼道路边缘检测过程

为了获得 3D 坑洞，通过以下步骤提取坑洞边缘周围的关键点。

1）首先基于共线方程计算图像中对应的点云像素，如式(17)所示。其中设图像中对应的像素坐标为(X_p, Y_p, Z_p)，共线方程上的点坐标为(x_p, y_p)。

$$\begin{cases} x_p - x_0 = -f\dfrac{a_1\left(X_p - X_s\right) + b_1\left(Y_p - Y_s\right) + c_1\left(Z_p - Z_s\right)}{a_3\left(X_p - X_s\right) + b_3\left(Y_p - Y_s\right) + c_3\left(Z_p - Z_s\right)} \\ y_p - y_0 = -f\dfrac{a_2\left(X_p - X_s\right) + b_2\left(Y_p - Y_s\right) + c_2\left(Z_p - Z_s\right)}{a_3\left(X_p - X_s\right) + b_3\left(Y_p - Y_s\right) + c_3\left(Z_p - Z_s\right)} \end{cases} \tag{17}$$

式 中，$(a_1, a_2, a_3, b_1, b_2, b_3, c_1, c_2, c_3, f)$为变换矩阵分量，可确定目标点的变换状态。

2）点分离后将选择目标外围边缘点以覆盖目标轮廓，以式(18)表示覆盖平面。

$$AX + BY + CZ + D = 0 \tag{18}$$

3）建立坑洼边缘估计方程，联立共线方程与覆盖平面得到式(19)。

$$\begin{cases} AX+BY+CZ+D=0 \\ X-X_s=(Z-Z_s)\dfrac{a_1(x-x_0)+a_2(y-y_0)-a_3 f}{c_1(x-x_0)+c_2(y-y_0)-c_3 f} \\ Y-Y_s=(Z-Z_s)\dfrac{b_1(x-x_0)+b_2(y-y_0)-b_3 f}{c_1(x-x_0)+c_2(y-y_0)-c_3 f} \end{cases} \tag{19}$$

4）基于式(19)推导，可以得到坑洞边缘的精确坐标如式(20)所示。

$$\begin{cases} X=\dfrac{B(k_bX_s-k_aY_s)+C(k_cZ_s-k_aY_s)-k_aD}{(k_aA+k_bB+k_cC)} \\ Y=\dfrac{A(k_aY_s-k_bX_s)+C(k_cY_s-k_bZ_s)-k_bD}{(k_aA+k_bB+k_cC)} \\ X=\dfrac{A(k_aZ_s-k_cX_s)+B(k_bZ_s-k_cY_s)-k_cD}{(k_aA+k_bB+k_cC)} \end{cases} \tag{20}$$

其中k_a、k_b、k_c的计算如式(21)所示。

$$\begin{cases} k_a=a_1(x-x_0)+a_2(y-y_0)-a_3 f \\ k_b=b_1(x-x_0)+b_2(y-y_0)-b_3 f \\ k_c=c_1(x-x_0)+c_2(y-y_0)-c_3 f \end{cases} \tag{21}$$

利用式（4）、（5），可以根据边缘在图像中的位置和由周围激光点描述的路面函数来计算候选坑洞的精确边缘。

在获取到全部边缘点后，采用高斯滤波遍历全部边缘目标点，计算目标点之间的空间欧式距离，通过欧式距离来计算对应的权重以区分远近目标。其表达式如式(22)所示。

$$f(x_i,x_j)=e^{x_i^T x_j} \tag{22}$$

考虑到实现难易，$e^{x_i^T x_j}$使用点乘相似性代替欧式距离，此时的归一化函数$C(x)$为$C(x)=\sum_{\forall j} f(x_i,x_j)$。

由于 Gaussian 是直接在当前空间进行计算，在嵌入空间中计算高斯距离，公式如式(23)所示。

$$f(x_i,x_j)=e^{\theta(x_i)^T\phi(x_j)} \tag{23}$$

其中$\theta(x_i)$、$\phi(x_j)$的计算如式(24)所示。

$$\begin{aligned} \theta(x_i)&=W_\theta x_i \\ \phi(x_j)&=W_\phi x_j \end{aligned} \tag{24}$$

高斯滤波流程如图 15 所示，提取图像三通道特征，然后将三维特征转化为二维特征，经过高斯滤波后将滤波特征值重组为三维向量输出。

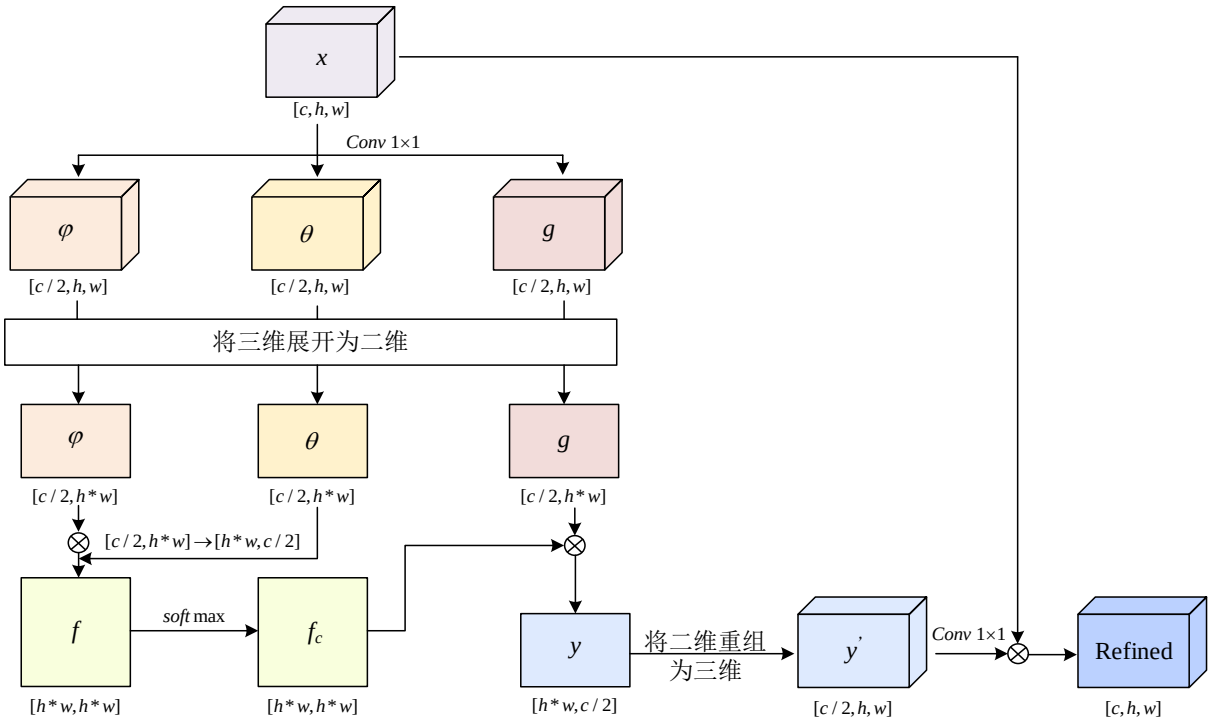


图 15 高斯滤波流程图

经过高斯滤波后将得到特征层下采样的四组特征值，输入至 YOLOv8 网络中进行特征解码以实验远距离坑洼道路识别。

6.3.4 道路检测网络特征提取

在获得道路目标位置后，需要提取其特征值，利用 Focus 结构可以保留目标的全局特征，结合超分辨率的对抗生成网络的细节特征，将有利于坑洼道路目标的检测。

首先需要对目标图像进行归一化，再基于 Focus 结构对目标图像进行切片，如图 16 所示，将 640×640×3 的 RGB 图像切片后得到四组 320×320×3 的特征图，再经过 32 个卷积核的 Conv 和 ReLu，最终得到 320×320×32 的特征图。

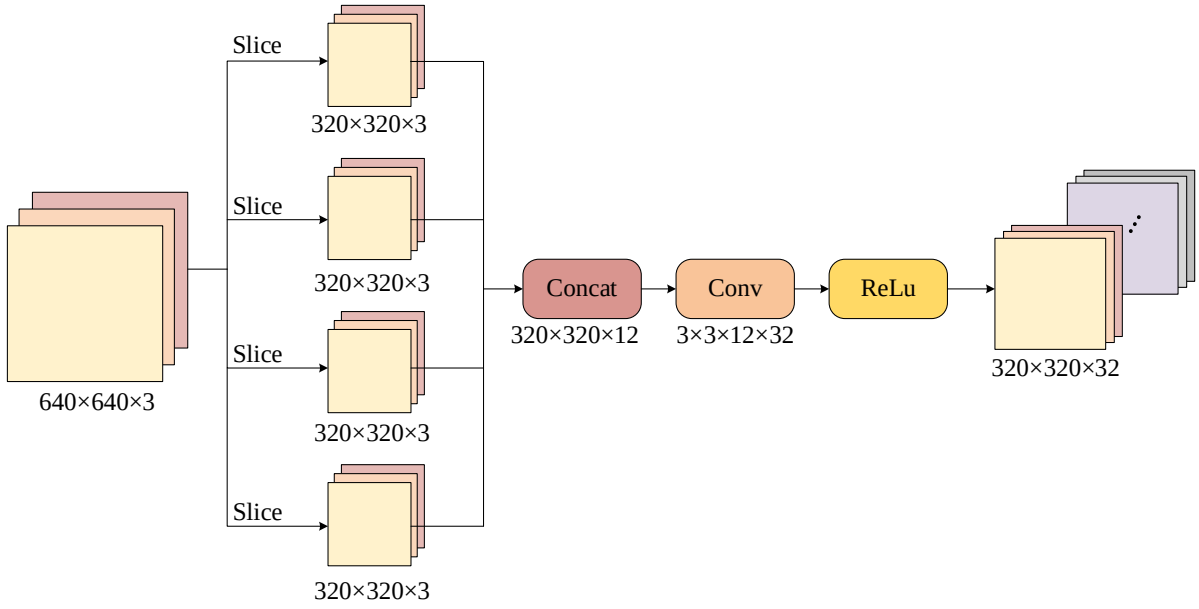


图 16 基于 Focus 结构的切片过程

在对 640×640×3 通道的图像进行特征提取时，切片后为 320×320×12 的特征图，再经过卷积后得到 320×320×32 的特征图，此时基于 Focus 结构得到的模型大小为 3×3×12×32×320×320=354M，而参数大小为 3×3×12×32+32+32=3520。若直接将图像

输入至识别网络中将得到的模型大小为 3×3×12×32×640×640+3×3×12×32×320×320=1769M，因此基于 Focus 结构对目标图像进行特征提取可以保留原有空间关系，还可以减小网络参数大小。

6.3.5 基于改进 IoU 的目标检测

在多数目标识别任务中，通常使用交并比（Intersection over Union, IoU）来衡量模型预测图片与真实目标的相似程度。通过计算预测结果记锚框与真实标注目标框的交集与并集的比值来得到 IoU，记锚框为 $B=[x\ y\ w\ h]$，目标框为 $B_{gt}=[x_{gt}\ y_{gt}\ w_{gt}\ h_{gt}]$，如图 17 所示。

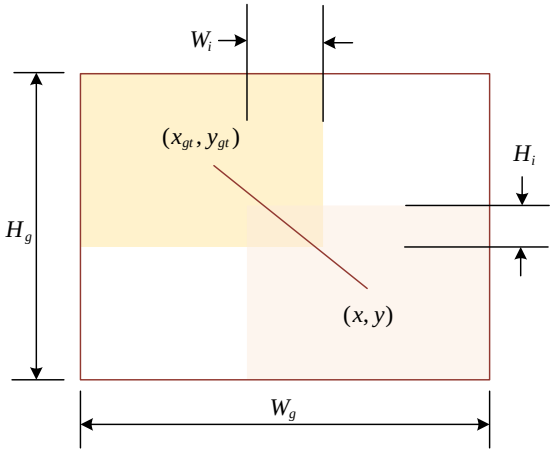


图 17 IoU 示意图

IoU 如式(25)所示。

$$\mathrm{L}_{IoU}=1-IoU=1-\frac{W_iH_i}{S_u} \tag{25}$$

但当边界框之间没有重叠时将会导致反向传播的随机下降梯度为0，如式(26)所示，进而导致无法训练损失。

$$\frac{\partial \mathrm{L}_{IoU}}{\partial W_i}=\begin{cases}-H_i\dfrac{IoU+1}{S_u}, W_i>0\\ 0, W_i=0\end{cases} \tag{26}$$

因此为解决训练中出现的误差，引入惩罚项来解决该问题，最终 IoU 将包含两项结果，如式(27)所示。

$$\mathrm{L}_i=\mathrm{L}_{IoU}+\Re_i \tag{27}$$

本文在 IoU 的基础上考虑了图片纵横比，保证预测框与目标框的一致性，如式(28)、(29)、(30)所示。

$$CIoU=IoU-\left(\frac{\rho^2\left(b,b^{gt}\right)}{c^2}+\alpha\upsilon\right) \tag{28}$$

$$\upsilon=\frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}}-\arctan\frac{w}{h}\right)^2 \tag{29}$$

$$\alpha=\begin{cases}0, IoU<0.5\\ \dfrac{\upsilon}{(1-IoU)+\upsilon}, IoU\geq 0.5\end{cases} \tag{30}$$

其中相关参数的计算如式(31)、(32)、(33)所示。

$$c\upsilon=\frac{4}{\pi^2}\left(\tan^{-1}\frac{w}{h}-\tan^{-1}\frac{w_{gt}}{h_{gt}}\right)^2 \tag{31}$$

$$\frac{\partial \upsilon}{\partial w}=\frac{8}{\pi^2}\left(\tan^{-1}\frac{w}{h}-\tan^{-1}\frac{w_{gt}}{h_{gt}}\right)^2\frac{h}{h^2+w^2} \tag{32}$$

$$\frac{\partial \upsilon}{\partial h}=-\frac{8}{\pi^2}\left(\tan^{-1}\frac{w}{h}-\tan^{-1}\frac{w_{gt}}{h_{gt}}\right)^2\frac{w}{h^2+w^2} \tag{33}$$

在式(33)中，υ 的反向传播梯度满足 $\frac{\partial \upsilon}{\partial h}=-\frac{w}{h}\frac{\partial \upsilon}{\partial w}$，这意味着 υ 无法为预测框的宽高提供同号的梯度。由于负梯度消失会导致预测框无法优化，但 CIoU 考虑了图像纵横比的一致性，在模型训练过程中利用 CIoU 自适应调整边界框损失并保持预测框与目标框的一致性，可以提高道路目标检测的准确性。

基于改进深度卷积核、特征增强模块、高斯滤波、共线方程以及 CIoU 等方法将得到如图 18 所示的改进 YOLOv8 模型结构，可进一步提升模型检测精度。

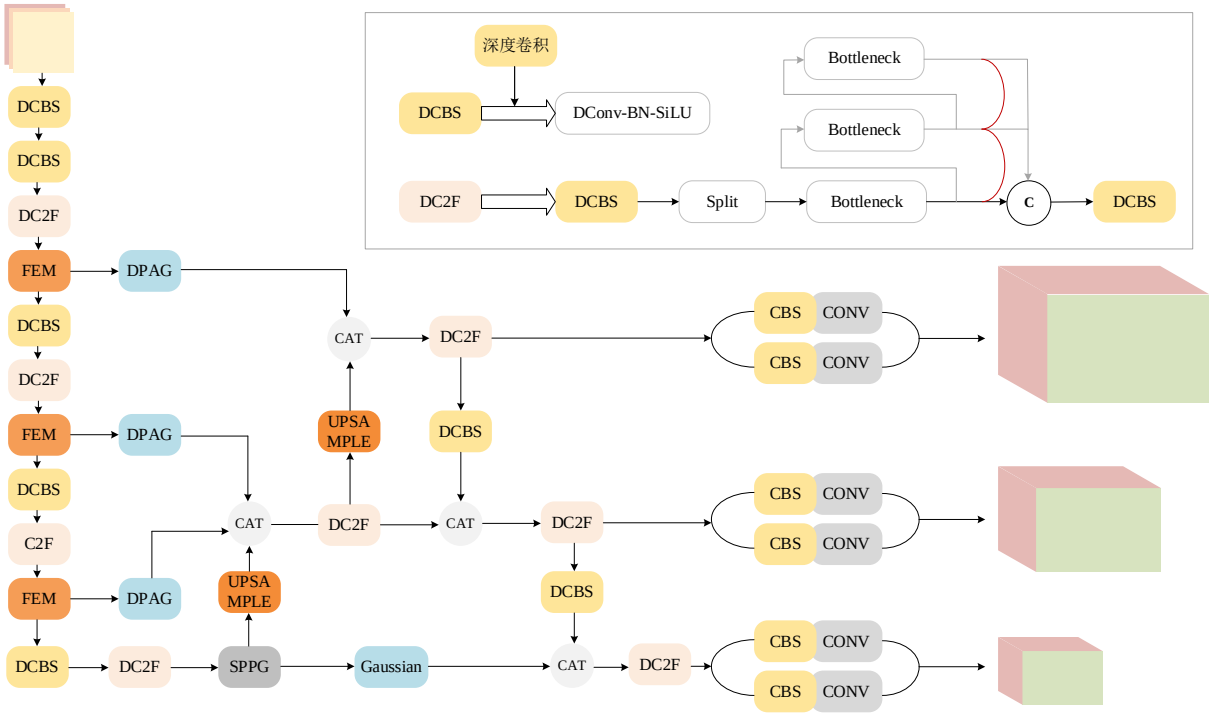


图 18 改进 YOLOv8 模型结构

6.4 YOLO 模型剪枝

在改进 YOLOv8 模型的基础上，可通过剪枝算法实现模型轻量化，去除不重要的神经元连接，有效减少模型参数量，如图 19 所示。通过对原始网络模型的第 i 层输入进行剪枝，去除部分卷积核，相应的特征图也会减少，进而导致后层卷积核也会减少，此时会致使模型性能下降，可将剪枝后的网络模型再训练以调整损失卷积核的权重，以此保持原有网络结构的性能。

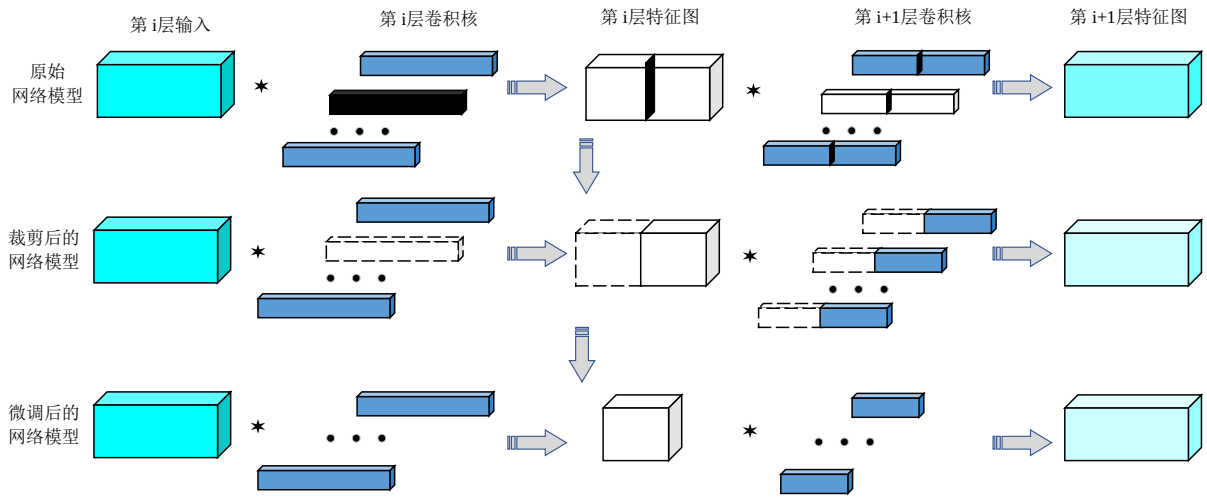


图 19 模型剪枝过程

在裁剪后的网络模型框架上，加上微调后的网络权重，可以在实现模型轻量化的同时达到原有网络模型的高精度检测。

6.5 模型测试

通过建立的模型可以提取目标特征值，如图 20 所示，保留坑洼目标的轮廓，以获取其图像特征。



图 20 目标特征图

除此之外，可通过热力图来获取坑洼位置的中心特征值，如图 21 所示。



图 21 目标热力图

问题一基于改进超分辨率对抗生成网络模型和改进 YOLOv8 模型最终可建立坑洼道路检测和识别模型提取图像特征，后续基于附件数据进行训练可实现快速响应、高识别率检测。

七、问题二模型建立与求解

为了验证问题一建立模型的有效性，问题二中将基于附件中所提供的图片数据进行训练，利用超分辨率生成对抗网络扩充训练集，对改进 YOLOv8 模型的模块进行消融实验，并且从模型收敛性和精度等方面进行评估。

7.1 训练参数

本次训练所使用的环境如表 2 所示，由于不同优化器对模型精度有影响，在训练过

程中将分别采用 SGD 和 Adam，比较二者性能优劣以获得最优权重。

表 2 训练参数

配置	参数	
CPU	Intel(R) Core(TM) i5-12400F 2.50 GHz×12	
GPU	NVIDIA GeForce RTX 3060	
输入图片分辨率	640×640×3	
学习框架	Pytorch	
优化器	SGD	Adam
epochs	500	300
学习率	0.05	0.01
batch_size	16	16

7.2 数据集简介

根据附件数据和 Labelimg 工具最终将得到如图 22 所示的 YOLO 数据集。后续将根据标签数据进行模型训练和图像生成，最终以得到高精度的权重文件，实现对测试数据的验证。



图 22 标记的坑洼道路图片

7.3 模型评价指标

仅对目标进行检测还无法准确评估模型的准确性，因此为了定量评价模型的有效性和精度，本文选取了准确率（Accuracy, Acc）、精确率（Precision, Pre）、召回率（Recall, Rec）、F1 分数（F1-score, Fsc）、平均精度（mean average precision, mAp）作为检测模型的评价指标。

1）准确率（Acc）计算公式如式(34)所示。

$$\mathrm{Acc}=\frac{n_{TP}+n_{TN}}{n_{TP}+n_{TN}+n_{FP}+n_{FN}} \tag{34}$$

其中，n_{TP}，n_{TN}，n_{FP} 以及 n_{FN} 分别代表真正、真负、假正和假负分类的数量。Acc 是指正确分类的比例，它与数据集的平衡有关，Acc 值对于模型非常有用。

2）精确率（Pre）计算公式如式(35)所示。

$$\mathrm{Pre}=\frac{n_{TP}}{n_{TP}+n_{FP}} \tag{35}$$

Pre 能够衡量实际正确的积极分类的比例，当 n_{FP} 的成本较高时 Pre 是一个很好的衡量标准。

3）召回率（Rec）计算公式如式(36)所示。

$$\mathrm{Rec}=\frac{n_{TP}}{n_{TP}+n_{FN}} \tag{36}$$

Rec 也称为灵敏度，是指检索到的相关实例的百分比。因此，精确度和召回率都以相关性为基础。精确度可视为质量的衡量标准，而召回率则是数量的衡量标准。

4）F1 分数（Fsc）计算公式如式(37)所示。

$$\mathrm{Fsc}=\frac{2\times n_{TP}}{2\times n_{TP}+n_{FP}+n_{FN}} \tag{37}$$

Fsc 是衡量模型在数据集上准确性的指标。它被定义为模型精度和召回率的调和平均值。当类分布相似时可以使用 Acc，而当存在不平衡类时 Fsc 是更好的度量。

5）平均精度（mAp）计算公式如式(38)所示。

$$mAp=\int_0^1 p(r)dr \tag{38}$$

mAp 是在 IoU 阈值下评估的精确度-召回曲线（ $p(r)$ ）下的面积。 $p(r)$ 是根据从数据集的整个图像获得的结果绘制的函数，x 轴表示 Rec，y 轴表示 Pre。

7.4 模型性能评估

7.4.1 收敛性分析

为了验证改进的 YOLOv8 模型在测试集上的收敛性能，本文比较了改进 YOLOv8 模型、改进 YOLOv8（SGD）、改进 YOLOv8（Adam）的收敛情况，三种情况下的损失和 mAp 随 epoch 的变化情况如图 23 和图 24 所示（统一 300epoch）。

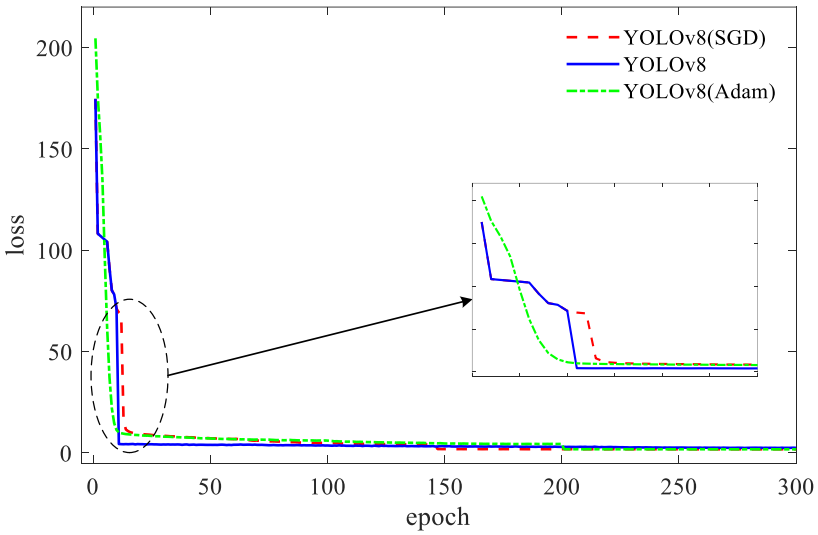


图 23 YOLOv8 模型训练损失变化曲线

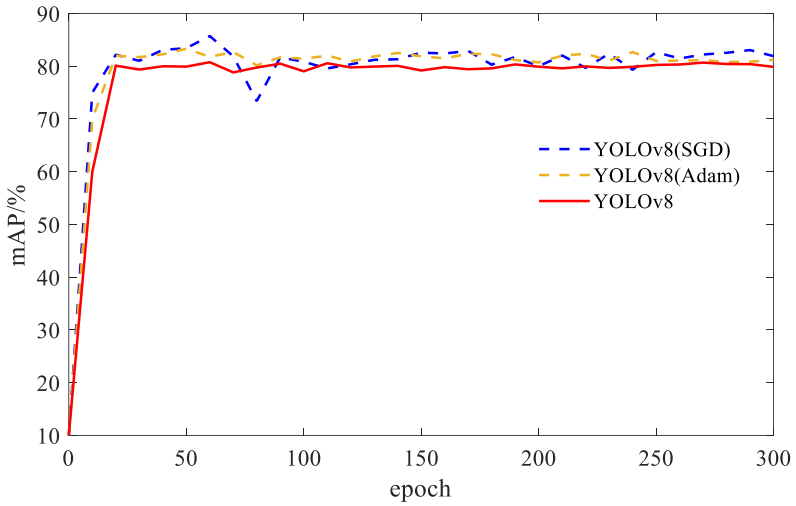


图 24 YOLOv8 模型训练 mAP 变化曲线

从图 23 整体上可以看出改进 YOLOv8、改进 YOLOv8（SGD）和改进 YOLOv8（Adam）模型的分类损失随 epoch 增加均能保持较低水平，且能达到快速收敛的效果，但聚焦局部可以看出通过 SGD 优化后的 YOLOv8 模型能更快收敛，而通过 Adam 优化后的 YOLOv8 模型收敛曲线更加平滑，从图 24 可以看出三种模型的 mAp 可以快速响应最终达到 80%左右，通过优化器的两组模型 mAp 达到平稳后稳定高于未经优化的 YOLOv8 模型，而经 SGD 优化后在 60epoch 时 mAp 达到最大，由此说明利用优化器可以加快模型收敛。

7.4.2 分类精度分析

为了进一步评估模型的检测精度，本文比较了改进 YOLOv8、改进 YOLOv8（SGD）、改进 YOLOv8（Adam）的精度，其中 Pre、Rec 以及 PR 曲线如图 25、图 26、图 27 所示。

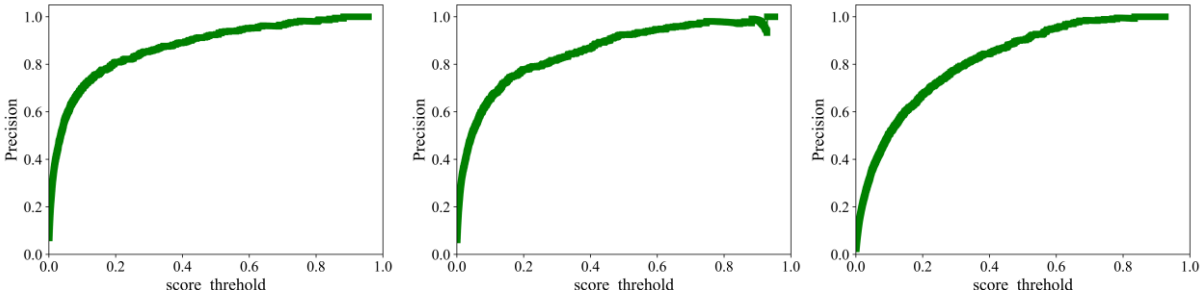


图 25 三组模型 Pre 曲线

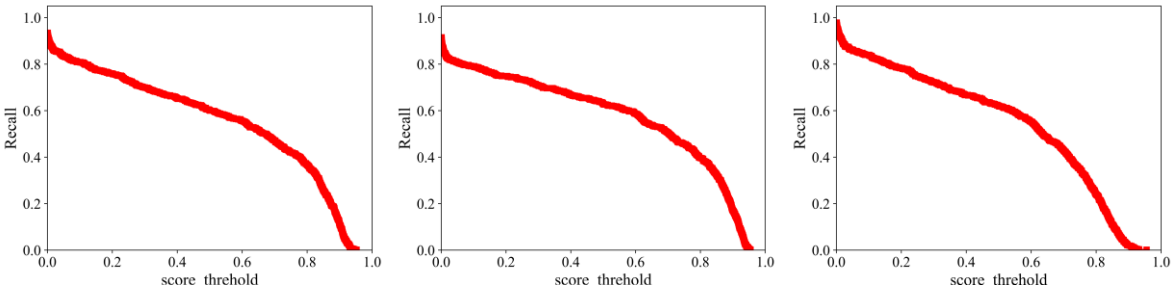


图 26 三组模型 Rec 曲线

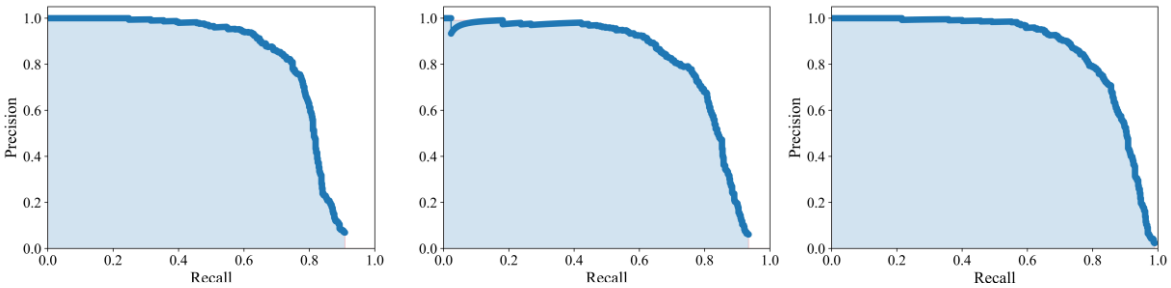


图 27 三组模型 PR 曲线

从图 25、图 26、图 27 中可以看出，通过优化器后的 YOLO 模型的精度相较于基础 YOLO 模型更高，三者的精度均高于 80%，且召回率超过 80%。从 PR 曲线中蓝色面积大小可以看出，经过优化器后的模型 Recall 更接近 100%，且利用 SGD 优化后的 YOLO 模型 PR 面积大于经 Adam 优化后的模型，即 mAp 值更大，说明模型预测精度更高。

综合模型收敛性和精度结果，基于 SGD 改进 YOLOv8 模型性能优于基于 Adam 改进 YOLOv8 模型，因此后续实验将基于 SGD 优化的改进 YOLOv8 模型权重进行测试。

7.5 消融实验

为验证本文模型改进的有效性，针对四点改进措施：引入 CBAM、基于 DCNN、Canny 边缘检测以及 CIoU，进行消融实验。得出各个模型的精确度 P、召回率 R 以及在不同 IoU 下的 mAP 值，具体结果如表 3 所示。

表 3 消融实验结果

模型方法					指标			
YOLOv8	CBAM	DCNN	Canny	CIoU	P/%	R/%	$mAP_{.5}$/%	$mAP_{.5:.95}$/%
√					82.4	59.6	72.3	40.2
√	√				85.7	60.1	76.1	44.9
√		√			86.3	61.4	78.5	45.8
√			√		84.2	58.9	74.3	42.3
√	√	√			90.9	68.2	81.4	48.3
√	√	√	√		91.2	71.3	82.1	49.5
√	√	√	√	√	93.3	74.8	86.6	52.4

根据表 3 可以看出，加入 CBAM、DCNN、Canny 和 CIoU 可以提升模型的性能，对比 YOLOv8 基准各方面均有提高，在 YOLOv8 的基础上引入 CBAM，精度提升了 3.2%，$mAP_{.5}$ 提升 3.8%，$mAP_{.5:.95}$ 提升 4.7%；在 YOLOv8 基础上改变卷积结构为 DCNN，精度提升 3.9%，$mAP_{.5}$ 提升 6.2%，$mAP_{.5:.95}$ 提升 5.6%；在 YOLOv8 基础上采用 Canny 边缘检测虽不如 CBAM 和 DCNN 性能好，但其精度相较于基准提升 1.8%，$mAP_{.5}$ 提升 1.0%，$mAP_{.5:.95}$ 提升 2.1%。最终将 CIoU 加入 YOLOv8-CBAM-DCNN-Canny 模型中，与 YOLOv8 相比，精度提升 10.9%，$mAP_{.5}$ 提升 14.2%，$mAP_{.5:.95}$ 提升 12.2%，根据以上数据分析可以得到，改变卷积核结构和 IoU、加入注意力机制和边缘检测方法对模型精度有较好的提升，在具体识别任务中可以达到快速响应。

7.6 模型对比实验

为进一步验证问题一提出的改进 YOLOv8 的有效性和泛化性能，将本文模型与 FCN[6]、Fast FCN、CNN、Fast RCNN、FPN[7]、ANN[8]、KNN、YOLOv5、YOLOv6、YOLOv7 等方法在同一数据集上进行对比分析，多种方法的精度对比结果如表 4 和图 28 所示。

表 4 多种检测方法精度对比结果

模型	参数量	性能指标				
		$mAP_{.5}$(%)	Fsc(%)	Pre(%)	Rec(%)	Acc(%)
FCN	50M	79.60	88.64	90.81	75.69	85.27
Fast FCN	88M	81.28	89.67	91.33	77.15	86.93
CNN	79M	78.29	88.04	89.97	73.16	84.26
Fast RCNN	102M	80.16	89.21	90.42	76.65	86.70
FPN	28M	77.19	87.10	90.91	83.59	86.61
ANN	46M	81.91	89.44	88.50	80.41	87.08
KNN	34M	79.36	88.83	90.66	70.17	87.04
YOLOv5	109M	80.46	89.40	91.22	80.97	92.60
YOLOv6	124M	79.70	89.14	90.31	80.62	91.88
YOLOv7	127M	76.33	89.61	82.75	69.11	88.36
YOLOv8	169M	72.30	89.37	82.40	59.63	85.37
改进 YOLOv8	84M	86.61	90.45	93.30	74.80	97.60

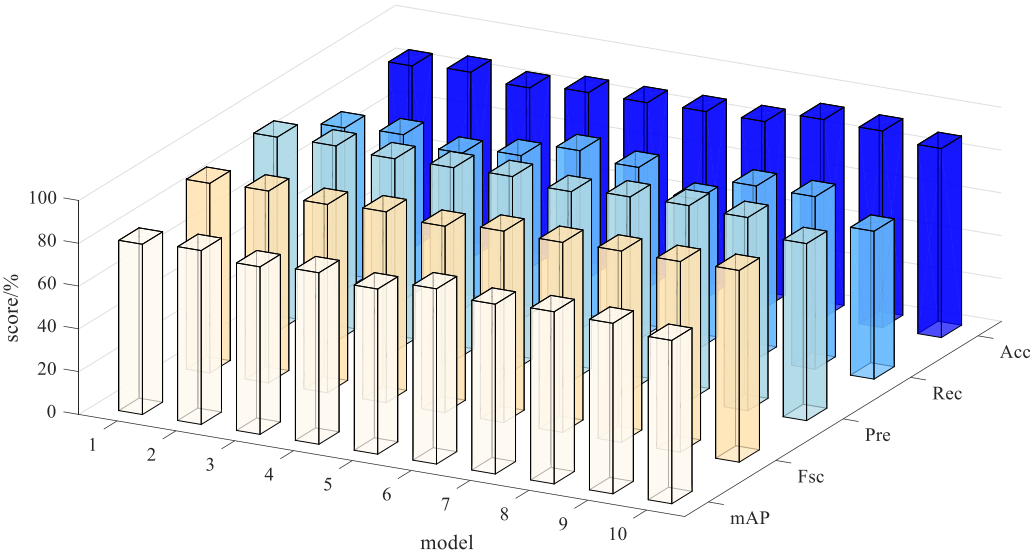


图 28 多种检测方法精度对比直方图

从表 4 和图 28 中可以看出，上述模型在对道路图像进行检测时可以保持较高的准确率，而本文建立的改进 YOLOv8 模型在 12 类检测模型中的准确率最高，其中 $mAP_{.5}$、Fsc、Pre、Acc 均高于其他模型，且 Acc 相较于 YOLOv8 提升了 12.23%，相较于应用广泛的 YOLOv5 提升了 5%，且精度明显优于单一检测模型如 FCN、CNN 等，但改进 YOLOv8 模型在召回率和参数量上远低于部分检测模型如 FPN、ANN，因为在提升精度的时候加重了模型网络结构，最终获得了高检测精度，但通过模型剪枝的方式可以降低其参数量，相较于 YOLOv5 至 YOLOv8 模型，改进后的 YOLOv8 模型参数量明显降低，表明了该模型的显著提升效果。

7.7 测试结果展示

在模型训练过程中，利用共线方程和高斯滤波边缘检测可以去除背景，只保留道路坑洼目标的轮廓，将有利于模型获取更加精确的特征值。基于附件训练集中的图像可以得到如图 29 所示的边缘检测结果。

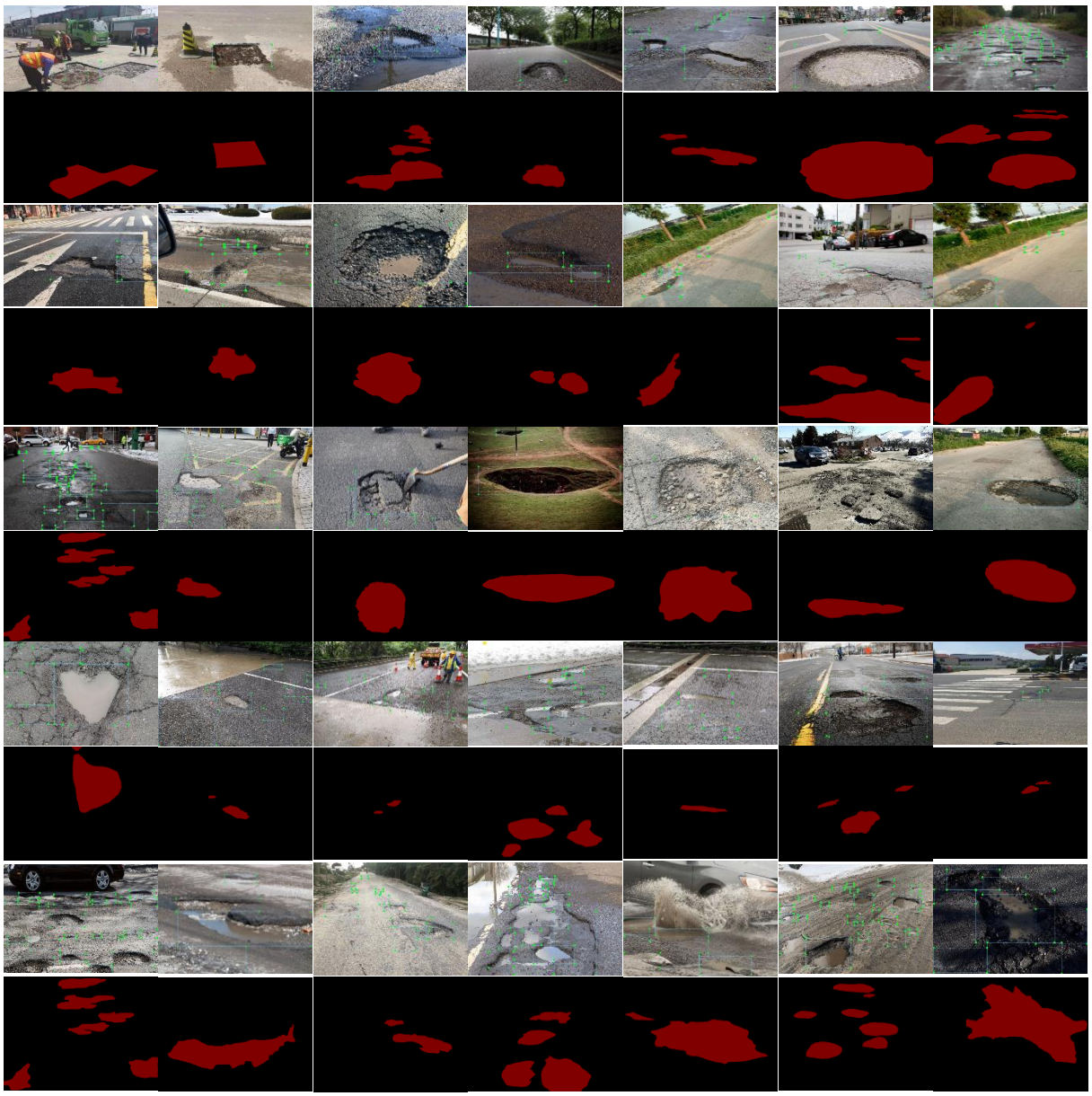


图 29 边缘检测结果

在获得边缘特征值后，利用生成对抗网络和深度卷积网络可以得到虚拟特征值，将其输入至 YOLO 网络中可以可以达到单目标和多目标的高精度识别，以部分图像为例，识别结果如图 30 和图 31 所示。



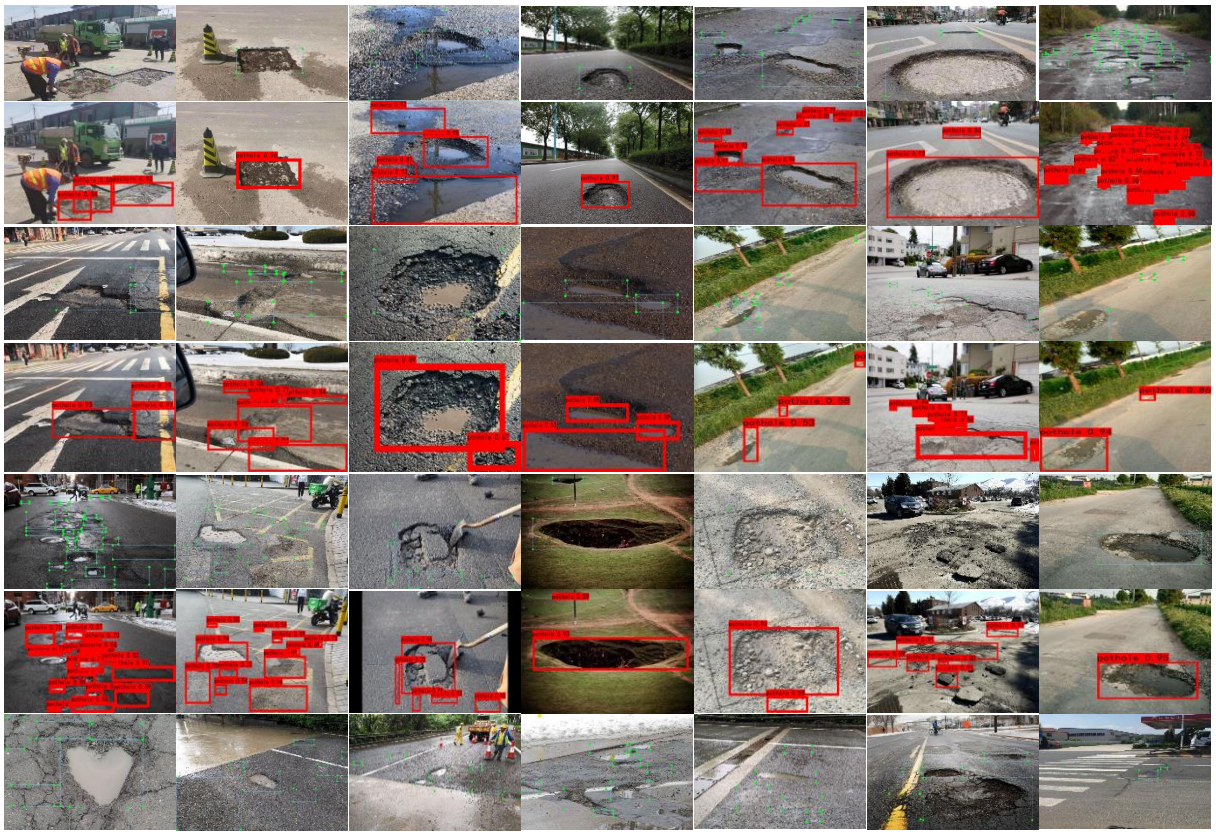


图 30 单目标识别结果



图 31 多目标识别结果

从图 30 和图 31 可以看出，改进的 YOLOv8 不仅可以检测出占据图片大部分比例的单一目标，还可以检测出远距离模糊、重叠小目标，且部分置信度可达到 90%以上。完整识别结果如图 32 所示。



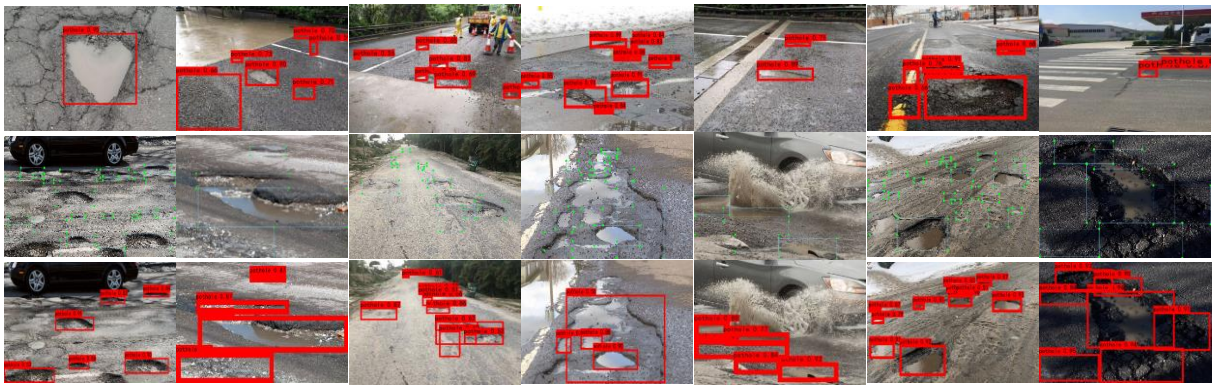


图 32 完整识别结果

通过对改进的 YOLOv8 模型的性能分析和实验结果展示，可以看出问题一模型的有效性与泛化性，在训练过程中可以达到快速收敛，在训练数据少的情况下利用生成对抗网络得到特征值可以提升特征数量，且优化后的模型相较于其他检测模型精度更高，可以达到高识别率、快速、分类准确的要求。

八、问题三模型建立与求解

8.1 基于改进 YOLOv8 模型的识别结果

根据问题一建立的道路识别模型，基于问题二训练的权重，将道路测试数据输入至检测识别网络中。对测试集的图片进行识别将得到如表所示的结果，表 5 列举前 10 张和后 10 张图片识别结果，完整识别结果见 test_result.csv，其中 1 表示正常道路，0 表示坑洼道路。

表 5 测试集道路识别结果

前 10 张		……	
frame	label	frame	label
00aqq56m.jpg	1	zx48xvxs.jpg	1
00c9sdfq.jpg	0	Zxfhwbki.jpg	0
00jjy9i3.jpg	1	zxgm3ar3.jpg	1
00q3q8py.jpg	1	zy7m9c8x.jpg	0
00yf8rxj.jpg	0	Zynixpux.jpg	0
0a9kzasw.jpg	0	zyylqfs0.jpg	0
0av740bq.jpg	0	zzhe6767.jpg	0
0b28dz29.jpg	0	zzipq9zn.jpg	0
0ba07d02.jpg	1	zzvom0ax.jpg	0
0bao2ebx.jpg	1	zzvth22q.jpg	1
……		后 10 张	

其中 testdata_V2 文件夹下包含 4942 张测试图片，通过改进 YOLOv8 模型将得到不同置信度的检测框识别结果，坑洼道路有 4045 张，正常道路有 897 张，如表 6 和图 33 所示。

表 6 置信度数量分布

置信度	＜0.5	0.5 -0.6	0.6 -0.7	0.7 -0.8	0.8 -0.9	＞0.9
数量	68	233	384	763	1958	639

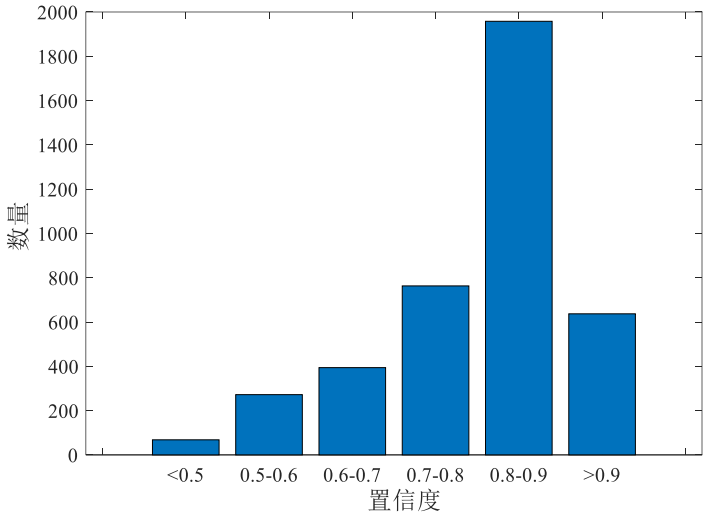


图 33 置信度分布数量直方图

从表 6 和图 33 可以看出，针对测试集中的图片检测置信度大多数可以达到 80%以上，甚至部分图片的识别置信度可达到 90%，虽出现少数目标置信度在 50%以下，但整张图片可以区分其是否为坑洼道路，仅检测精度有所下降，可以实现对道路坑洼的识别。

8.2 基于改进 YOLOv8 模型的检测结果

本文将测试图像输入至网络中，以热力图的方式反映图像中坑洼位置的预测情况和特征图，模型将目标看成一个点进行检测，热力图则表示分类信息，坑洼位置处对应一个热力点，若原图像中有目标的中心点，将会在该中心点处产生一个热力点，如图 34 所示，图 列举了部分道路识别后的热力图，通过热力图的形式有助于提取特征图，且可以快速定位图中哪部分主导最终的识别结果。图中所示的热力结果均能反映检测目标的位置，便于检测框获取目标中心点的位置。



图 34 检测热力图

将测试数据全部输入到预训练模型中，可以基于训练权重得到预测结果，针对图中单个坑洞目标的图像检测可以达到较高精度，如图 35 所示，图 35 列举部分单目标识别结果，由于图中无相似目标进行干扰，多数目标的置信度可以达到 90%以上，以此说明了模型具有较高的检测精度。

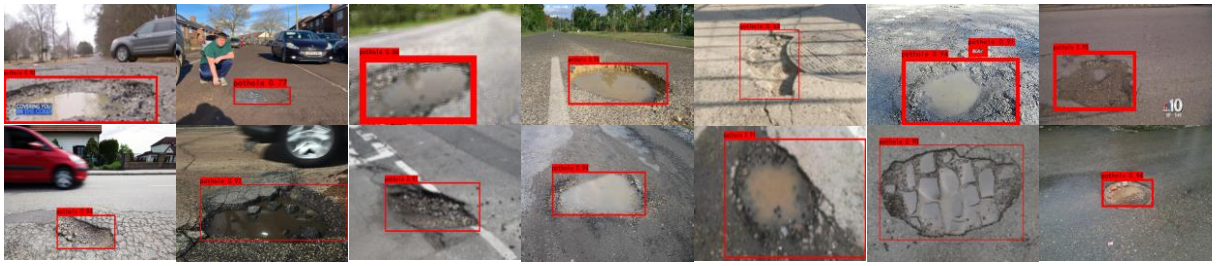




图 35 改进 YOLOv8 检测结果（单目标）

同理，将测试集输入至检测网络中，针对多目标远距离、重叠遮挡情况，采用问题一模型依然可以达到较好的检测效果，如图 36 所示，图 36 列举部分多目标检测结果，图像中出现多个坑洼位置时，将会优先标记具有强特征的目标，然后再检测其周围目标点，虽然少数小目标置信度较低，但部分结果对图中每个目标的置信度仍可达到 80%以上，说明了模型检测的有效性。





图 36 改进 YOLOv8 检测结果（多目标）

根据以上实验结果，本文建立的模型可以实现对测试集的目标检测和识别，且对远距离、遮挡情况下的目标也有较好的检测效果，且检测置信度高，表明了模型的有效性，可以基于计算机视觉实验坑洼道路检测和识别。

九、模型评价

9.1 模型优点

9.1.1 问题一优点

1）采用改进的超分辨率对抗生成网络以特征生成的方式在原有数据集的基础上扩充目标特征图，基于生成网络和对抗网络可以得到接近于原始图像的特征图，采用注意力机制增强道路坑洼目标与环境的耦合性，进而提升了生成特征值的置信度。

2）采用改进的 YOLOv8 检测网络以实现快速识别、高精度检测分类的效果，通过改进深度卷积网络，加入增强特征模块和 Canny 边缘检测达到对道路坑洼位置的高精度检测。

3）采用模型图像分块和模型剪枝的方法降低整个检测网络的参数量，在原有基础上实现轻量化。

9.1.2 问题二优点

1）针对问题一提出的检测模型，通过多个评价指标从收敛性和精度等维度进行评估，且具有达到较好的评价效果。

2）通过消融实验验证了模型改进对检测精度的提升，并通过与多种检测算法的对比，验证了改进 YOLOv8 模型的高精度。

9.1.3 问题三优点

基于问题一检测模型与问题二的预训练权重得到测试集下的检测结果，保持了整体建模的连贯性，模型可以准确识别坑洼道路，且多数目标可以达到 80%以上的置信度。

9.2 模型缺点

虽然本文建立的模型具有较高的检测精度，但不免存在一些问题，可以从以下方面进行考虑。

1）本文提出的检测模型由于训练样本数量少，基于特征生成的方式扩充目标特征仅仅适用于小样本权重，后续可考虑增加训练样本数，进一步提升模型精度。

2）由于自然场景中的图像较为复杂，获取的图像可能存在低分辨率、重复遮挡等情况，对多个目标的检测精度还有待提升，可以考虑优化模型结构，提高多目标检测的置信度。

参考文献

[1] X Wang, K Yu, S Wu, et al. ESRGAN: Enhanced super-resolution generative adversarial networks[C]//L Leal Taixe, S Roth. Computer Vision - Eccv 2018 Workshops, Pt V: Vol. 11133. Cham: Springer International Publishing Ag, 11133: 63-79, 2019.

[2] WOO S, PARK J, LEE J Y, et al. Cbam: Convolutional block attention module[C]. Proceedings of the European conference on computer vision (ECCV). 11211: 3-19, 2018.

[3] Karna N B A, Putra M A P, Rachmawati S M, et al. Toward accurate fused deposition modeling 3D printer fault detection using improved YOLOv8 with hyperparameter optimization[J]. IEEE ACCESS, 11:74251-74262, 2023.

[4] 李明珏, 张高兴. 基于 YOLO 目标检测的边缘到终端的道路损伤检测系统设计[J]. 机电工程技术, 52(9): 291-295, 2023.

[5] Reis F M C V, Liu J, Schuette P J , et al. Shared dorsal periaqueductal gray activation patterns during exposure to innate and conditioned threats[J]. The Journal of Neuroscience, 41(25): 5399-5420, 2021.

[6] J. Long, E. Shelhamer, T. Darrell. Fully convolutional networks for semantic segmentation[C]. in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 3431-3440, 2015.

[7] A Kirillov, R. Girshick, K. He, et al. Panoptic feature pyramid networks[C]. in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition(CVPR), 6399-6408, 2019.

[8] Z Zhu, M. Xu, S. Bai, et al. Asymmetric Non-Local neural networks for semantic segmentation[C]. in Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). 593-602, 2019.

附录

支撑材料清单：

文件夹 1：img 存储训练图片与测试图片
文件夹 2：img_out 存储检测图片
文件夹 3：logs 存放权重文件
文件夹 4：map_out 存放输出结果
文件夹 5：model_data 存放模型结构和热力图结果
文件夹 6：nets 存放 YOLO 检测主网络
文件夹 7：utils 存放 YOLO 检测绘图文件
文件夹 8：utils_coco 存放获取标签数据程序
文件夹 9：VOCdevkit 存放标签数据和图片

附件 1：FancyBar3.m 绘图代码
附件 2：get_map.py 绘制训练过程图代码
附件 3：plot_figure.m 绘图代码
附件 4：predict.py 检测网络预测代码
附件 5：requirements.txt 依赖环境
附件 6：summary.py 查看网络结构代码
附件 7：train.py 检测模型训练代码
附件 8：voc_annotation.py 检测模型读取标签和划分测试验证集代码
附件 9：yolo.py 检测网络主干代码

程序编号	1	程序说明	train.py（模型训练）	编程软件	Python

```
# 对数据集进行训练
import datetime
from functools import partial
import numpy as np
import torch
import torch.backends.cudnn as cudnn
import torch.distributed as dist
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from nets.yolo import YoloBody
from nets.yolo_training import (Loss, ModelEMA, get_lr_scheduler,
                                set_optimizer_lr, weights_init)
from utils.callbacks import EvalCallback, LossHistory
from utils.dataloader import YoloDataset, yolo_dataset_collate
from utils.utils import (download_weights, get_classes, seed_everything,
                         show_config, worker_init_fn)
from utils.utils_fit import fit_one_epoch
import os
os.environ['KMP_DUPLICATE_LIB_OK']='TRUE'


if __name__ == "__main__":
    #---------------------------------#
    #   Cuda    是否使用 Cuda
    #           没有 GPU 可以设置成 False
    Cuda = True
    seed = 11
    distributed = False
    #   sync_bn     是否使用 sync_bn，DDP 模式多卡可用
    sync_bn = False
    #   fp16        是否使用混合精度训练
    fp16 = False
    classes_path = 'model_data/voc_classes.txt'

    model_path = 'logs/ep1000-loss1.892-val_loss3.867.pth'
    #   input_shape     输入的 shape 大小，一定要是 32 的倍数
    input_shape = [640, 640]
    phi = 's'
    pretrained = False
    mosaic = True
    mosaic_prob = 0.5
    mixup = True
    mixup_prob = 0.5
    special_aug_ratio = 0.7
    #   label_smoothing     标签平滑。一般 0.01 以下。如 0.01、0.005。
    label_smoothing = 0
    Init_Epoch = 0
    Freeze_Epoch = 250
    Freeze_batch_size    = 16
    UnFreeze_Epoch = 500
    Unfreeze_batch_size = 16
    Freeze_Train = True
    Init_lr = 1e-2
    Min_lr = Init_lr * 0.01
```

```
optimizer_type = "sgd"
momentum = 0.937
weight_decay = 5e-4
#     lr_decay_type     使用到的学习率下降方式，可选的有 step、cos
lr_decay_type = "cos"
#     save_period         多少个 epoch 保存一次权值
save_period = 5
#     save_dir              权值与日志文件保存的文件夹
save_dir = 'logs'
eval_flag = True
eval_period = 10
num_workers = 4
#     train_annotation_path     训练图片路径和标签
#     val_annotation_path         验证图片路径和标签
train_annotation_path = '2007_train.txt'
val_annotation_path = '2007_val.txt'
seed_everything(seed)
ngpus_per_node = torch.cuda.device_count()
if distributed:
    dist.init_process_group(backend="nccl")
    local_rank = int(os.environ["LOCAL_RANK"])
    rank = int(os.environ["RANK"])
    device = torch.device("cuda", local_rank)
    if local_rank == 0:
        print(f"[{os.getpid()}] (rank = {rank}, local_rank = {local_rank}) training...")
        print("Gpu Device Count : ", ngpus_per_node)
else:
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    local_rank = 0
    rank = 0
class_names, num_classes = get_classes(classes_path)
if pretrained:
    if distributed:
        if local_rank == 0:
            download_weights(phi)
        dist.barrier()
    else:
        download_weights(phi)

#------------------------------------------------------#
#     创建 yolo 模型
#------------------------------------------------------#
model = YoloBody(input_shape, num_classes, phi, pretrained=pretrained)

if model_path != '':
    if local_rank == 0:
        print('Load weights {}.'.format(model_path))
    #     根据预训练权重的 Key 和模型的 Key 进行加载
    model_dict = model.state_dict()
    pretrained_dict = torch.load(model_path, map_location = device)
    load_key, no_load_key, temp_dict = [], [], {}
    for k, v in pretrained_dict.items():
        if k in model_dict.keys() and np.shape(model_dict[k]) == np.shape(v):
            temp_dict[k] = v
            load_key.append(k)
        else:
            no_load_key.append(k)
    model_dict.update(temp_dict)
    model.load_state_dict(model_dict)
    #------------------------------------------------------#
```

```
            #   显示没有匹配上的 Key
            #---------------------------------------------------#
            if local_rank == 0:
                print("\nSuccessful Load Key:", str(load_key)[:500], "……\nSuccessful Load Key Num:", len(load_key))
                print("\nFail To Load Key:", str(no_load_key)[:500], "……\nFail To Load Key num:", len(no_load_key))
                print("\n\033[1;33;44m 温馨提示，head 部分没有载入是正常现象，Backbone 部分没有载入是错误的。\033[0m")

        #   获得损失函数
        yolo_loss = Loss(model)
        #   记录 Loss
        if local_rank == 0:
            time_str = datetime.datetime.strftime(datetime.datetime.now(),'%Y_%m_%d_%H_%M_%S')
            log_dir = os.path.join(save_dir, "loss_" + str(time_str))
            loss_history = LossHistory(log_dir, model, input_shape=input_shape)
        else:
            loss_history = None

        if fp16:
            from torch.cuda.amp import GradScaler as GradScaler
            scaler = GradScaler()
        else:
            scaler = None

        model_train = model.train()
        #----------------------------#
        #   多卡同步 Bn
        #----------------------------#
        if sync_bn and ngpus_per_node > 1 and distributed:
            model_train = torch.nn.SyncBatchNorm.convert_sync_batchnorm(model_train)
        elif sync_bn:
            print("Sync_bn is not support in one gpu or not distributed.")

        if Cuda:
            if distributed:
                #----------------------------#
                #   多卡平行运行
                #----------------------------#
                model_train = model_train.cuda(local_rank)
                model_train = torch.nn.parallel.DistributedDataParallel(model_train, device_ids=[local_rank], find_unused_parameters=True)
            else:
                model_train = torch.nn.DataParallel(model)
                cudnn.benchmark = True
                model_train = model_train.cuda()

        #   权值平滑
        ema = ModelEMA(model_train)
        #---------------------------#
        #   读取数据集对应的 txt
        #---------------------------#
        with open(train_annotation_path, encoding='utf-8') as f:
            train_lines = f.readlines()
        with open(val_annotation_path, encoding='utf-8') as f:
            val_lines = f.readlines()
        num_train = len(train_lines)
        num_val = len(val_lines)

        if local_rank == 0:
            show_config(
```

```
            classes_path = classes_path, model_path = model_path, input_shape = input_shape, \
            Init_Epoch = Init_Epoch, Freeze_Epoch = Freeze_Epoch, UnFreeze_Epoch = UnFreeze_Epoch, Freeze_batch_size = Freeze_batch_size, Unfreeze_batch_size = Unfreeze_batch_size, Freeze_Train = Freeze_Train, \
            Init_lr = Init_lr, Min_lr = Min_lr, optimizer_type = optimizer_type, momentum = momentum, lr_decay_type = lr_decay_type, \
            save_period = save_period, save_dir = save_dir, num_workers = num_workers, num_train = num_train, num_val = num_val
        )
        wanted_step = 5e4 if optimizer_type == "sgd" else 1.5e4
        total_step  = num_train // Unfreeze_batch_size * UnFreeze_Epoch
        if total_step <= wanted_step:
            if num_train // Unfreeze_batch_size == 0:
                raise ValueError('数据集过小，无法进行训练，请扩充数据集。')
            wanted_epoch = wanted_step // (num_train // Unfreeze_batch_size) + 1
            print("\n\033[1;33;44m[Warning] 使用%s 优化器时，建议将训练总步长设置到%d 以上。\033[0m"%(optimizer_type, wanted_step))
            print("\033[1;33;44m[Warning] 本次运行的总训练数据量为%d，Unfreeze_batch_size 为%d，共训练%d个 Epoch，计算出总训练步长为%d。\033[0m"%(num_train, Unfreeze_batch_size, UnFreeze_Epoch, total_step))
            print("\033[1;33;44m[Warning] 由于总训练步长为%d，小于建议总步长%d，建议设置总世代为%d。\033[0m"%(total_step, wanted_step, wanted_epoch))

    #   主干特征提取网络特征通用，冻结训练可以加快训练速度
    if True:
        UnFreeze_flag = False
        #   冻结一定部分训练
        if Freeze_Train:
            for param in model.backbone.parameters():
                param.requires_grad = False

        batch_size = Freeze_batch_size if Freeze_Train else Unfreeze_batch_size

        #-------------------------------------------------------------------#
        #   判断当前 batch_size，自适应调整学习率
        #-------------------------------------------------------------------#
        nbs = 64
        lr_limit_max = 1e-3 if optimizer_type == 'adam' else 5e-2
        lr_limit_min = 3e-4 if optimizer_type == 'adam' else 5e-4
        Init_lr_fit = min(max(batch_size / nbs * Init_lr, lr_limit_min), lr_limit_max)
        Min_lr_fit = min(max(batch_size / nbs * Min_lr, lr_limit_min * 1e-2), lr_limit_max * 1e-2)

        #---------------------------------------#
        #   根据 optimizer_type 选择优化器
        #---------------------------------------#
        pg0, pg1, pg2 = [], [], []
        for k, v in model.named_modules():
            if hasattr(v, "bias") and isinstance(v.bias, nn.Parameter):
                pg2.append(v.bias)
            if isinstance(v, nn.BatchNorm2d) or "bn" in k:
                pg0.append(v.weight)
            elif hasattr(v, "weight") and isinstance(v.weight, nn.Parameter):
                pg1.append(v.weight)
        optimizer = {
            'adam': optim.Adam(pg0, Init_lr_fit, betas = (momentum, 0.999)),
            'sgd': optim.SGD(pg0, Init_lr_fit, momentum = momentum, nesterov=True)
        }[optimizer_type]
        optimizer.add_param_group({"params": pg1, "weight_decay": weight_decay})
        optimizer.add_param_group({"params": pg2})
        #   获得学习率下降的公式
        lr_scheduler_func = get_lr_scheduler(lr_decay_type, Init_lr_fit, Min_lr_fit, UnFreeze_Epoch)
```

```
        epoch_step = num_train // batch_size
        epoch_step_val   = num_val // batch_size

        if epoch_step == 0 or epoch_step_val == 0:
            raise ValueError("数据集过小，无法继续进行训练，请扩充数据集。")

        if ema:
            ema.updates = epoch_step * Init_Epoch

        #   构建数据集加载器。
        train_dataset = YoloDataset(train_lines, input_shape, num_classes, epoch_length=UnFreeze_Epoch, \
                                    mosaic=mosaic, mixup=mixup, mosaic_prob=mosaic_prob,
mixup_prob=mixup_prob, train=True, special_aug_ratio=special_aug_ratio)
        val_dataset = YoloDataset(val_lines, input_shape, num_classes, epoch_length=UnFreeze_Epoch, \
                                    mosaic=False, mixup=False, mosaic_prob=0, mixup_prob=0,
train=False, special_aug_ratio=0)

        if distributed:
            train_sampler = torch.utils.data.distributed.DistributedSampler(train_dataset, shuffle=True,)
            val_sampler = torch.utils.data.distributed.DistributedSampler(val_dataset, shuffle=False,)
            batch_size = batch_size // ngpus_per_node
            shuffle = False
        else:
            train_sampler = None
            val_sampler = None
            shuffle = True
        gen = DataLoader(train_dataset, shuffle = shuffle, batch_size = batch_size, num_workers = num_workers,
pin_memory=True,
                         drop_last=True, collate_fn=yolo_dataset_collate, sampler=train_sampler,
                         worker_init_fn=partial(worker_init_fn, rank=rank, seed=seed))
        gen_val = DataLoader(val_dataset  , shuffle = shuffle, batch_size = batch_size, num_workers = num_workers,
pin_memory=True,
                         drop_last=True, collate_fn=yolo_dataset_collate, sampler=val_sampler,
                         worker_init_fn=partial(worker_init_fn, rank=rank, seed=seed))
        #   记录 eval 的 map 曲线
        if local_rank == 0:
            eval_callback   = EvalCallback(model, input_shape, class_names, num_classes, val_lines, log_dir, Cuda, \
                                           eval_flag=eval_flag, period=eval_period)
        else:
            eval_callback   = None
        #   开始模型训练
        for epoch in range(Init_Epoch, UnFreeze_Epoch):
            #---------------------------------------#
            #   如果模型有冻结学习部分
            #   则解冻，并设置参数
            #---------------------------------------#
            if epoch >= Freeze_Epoch and not UnFreeze_flag and Freeze_Train:
                batch_size = Unfreeze_batch_size
                #-------------------------------------------------------------------#
                #   判断当前 batch_size，自适应调整学习率
                #-------------------------------------------------------------------#
                nbs   = 64
                lr_limit_max = 1e-3 if optimizer_type == 'adam' else 5e-2
                lr_limit_min= 3e-4 if optimizer_type == 'adam' else 5e-4
                Init_lr_fit= min(max(batch_size / nbs * Init_lr, lr_limit_min), lr_limit_max)
                Min_lr_fit= min(max(batch_size / nbs * Min_lr, lr_limit_min * 1e-2), lr_limit_max * 1e-2)
                #   获得学习率下降的公式
                lr_scheduler_func = get_lr_scheduler(lr_decay_type, Init_lr_fit, Min_lr_fit, UnFreeze_Epoch)
                for param in model.backbone.parameters():
                    param.requires_grad = True
```

```
                    epoch_step          = num_train // batch_size
                    epoch_step_val    = num_val // batch_size
                    if epoch_step == 0 or epoch_step_val == 0:
                        raise ValueError("num Error")
                    if ema:
                        ema.updates         = epoch_step * epoch
                    if distributed:
                        batch_size    = batch_size // ngpus_per_node
                    gen=  DataLoader(train_dataset,  shuffle  =  shuffle,  batch_size  =  batch_size,  num_workers  =
num_workers,       pin_memory=True,drop_last=True,       collate_fn=yolo_dataset_collate,       sampler=train_sampler,
worker_init_fn=partial(worker_init_fn, rank=rank, seed=seed))
                    gen_val=  DataLoader(val_dataset    ,  shuffle  =  shuffle,  batch_size  =  batch_size,  num_workers  =
num_workers,      pin_memory=True,      drop_last=True,      collate_fn=yolo_dataset_collate,      sampler=val_sampler,
worker_init_fn=partial(worker_init_fn, rank=rank, seed=seed))
                    UnFreeze_flag     = True
                gen.dataset.epoch_now            = epoch
                gen_val.dataset.epoch_now     = epoch
                if distributed:
                    train_sampler.set_epoch(epoch)
                set_optimizer_lr(optimizer, lr_scheduler_func, epoch)
                fit_one_epoch(model_train,  model,  ema,  yolo_loss,  loss_history,  eval_callback,  optimizer,  epoch,
epoch_step, epoch_step_val, gen, gen_val, UnFreeze_Epoch, Cuda, fp16, scaler, save_period, save_dir, local_rank)
                if distributed:
                    dist.barrier()
            if local_rank == 0:
                loss_history.writer.close()
```

程序编号	2	程序说明	predict.py（预测）	编程软件	Python

```
# 通过指定 mode 进行模式的修改。
import time
import cv2
import numpy as np
import os
from PIL import Image
from yolo import YOLO

os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'
if __name__ == "__main__":
    yolo = YOLO()
    mode = "dir_predict"
    crop = False
    count = False
    video_path = 0
    video_save_path = ""
    video_fps = 25.0
    test_interval = 100
    fps_image_path = "img/potholes00003.jpg"
    dir_origin_path = "img/"
    dir_save_path = "./img_out/test1"
    heatmap_save_path = "model_data/heatmap"
```

```
    simplify = True
    onnx_save_path = "model_data/models.onnx"
    if mode == "predict":
        while True:
            img = input('Input image filename:')
            try:
                image = Image.open(img)
            except:
                print('Open Error! Try again!')
                continue
            else:
                r_image = yolo.detect_image(image, crop=crop, count=count)
                r_image.show()
    elif mode == "video":
        capture = cv2.VideoCapture(video_path)
        if video_save_path != "":
fourcc=cv2.VideoWriter_fourcc(*'XVID')size=(int(capture.get(cv2.CAP_PROP_FRAME_WIDTH)),int(capture.get(cv2.
CAP_PROP_FRAME_HEIGHT)))
            out = cv2.VideoWriter(video_save_path, fourcc, video_fps, size)
        ref, frame = capture.read()
        if not ref:
            raise ValueError(0)
        fps = 0.0
        while (True):
            t1 = time.time()
            # 读取某一帧
            ref, frame = capture.read()
            if not ref:
                break
            # 格式转变，BGRtoRGB
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            # 转变成 Image
            frame = Image.fromarray(np.uint8(frame))
            # 进行检测
            frame = np.array(yolo.detect_image(frame))
            frame = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
            fps = (fps + (1. / (time.time() - t1))) / 2
            print("fps= %.2f" % (fps))
            frame=cv2.putText(frame,"fps=%.2f" % (fps), (0, 40), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
            cv2.imshow("video", frame)
            c = cv2.waitKey(1) & 0xff
            if video_save_path != "":
                out.write(frame)
            if c == 27:
                capture.release()
```

```
                    break
            print("Video Detection Done!")
            capture.release()
            if video_save_path != "":
                print("Save processed video to the path :" + video_save_path)
                out.release()
            cv2.destroyAllWindows()
        elif mode == "fps":
            img = Image.open(fps_image_path)
            tact_time = yolo.get_FPS(img, test_interval)
            print(str(tact_time) + ' seconds, ' + str(1 / tact_time) + 'FPS, @batch_size 1')
        elif mode == "dir_predict":
            from tqdm import tqdm
            img_names = os.listdir(dir_origin_path)
            for img_name in tqdm(img_names):
                if img_name.lower().endswith(
                        ('.bmp', '.dib', '.png', '.jpg', '.jpeg', '.pbm', '.pgm', '.ppm', '.tif', '.tiff')):
                    image_path = os.path.join(dir_origin_path, img_name)
                    image = Image.open(image_path)
                    r_image = yolo.detect_image(image)
                    if not os.path.exists(dir_save_path):
                        os.makedirs(dir_save_path)
                    print(os.path.abspath(os.path.join(dir_save_path,img_name.replace(".jpg", ".png"))))
                    with open("C:/Users/Administrator/Desktop/mathorcup/groundname2.txt", "a") as f:
                        f.write(str(os.path.abspath(os.path.join(dir_save_path, img_name.replace(".jpg", ".png")))) + '\n')
                    r_image.save(os.path.join(dir_save_path, img_name.replace(".jpg", ".png")), quality=95,
subsampling=0)
        elif mode == "heatmap":
            while True:
                img = input('Input image filename:')
                try:
                    image = Image.open(img)
                except:
                    print('Open Error! Try again!')
                    continue
                else:
                    yolo.detect_heatmap(image, heatmap_save_path)
        elif mode == "export_onnx":
            yolo.convert_to_onnx(simplify, onnx_save_path)
        else:
            raise AssertionError("Error")
```

程序编号	3	程序说明	yolo.py（检测框架）	编程软件	Python

```
import numpy as np
```

```
import torch
import torch.nn as nn
from PIL import ImageDraw, ImageFont
import colorsys
import os
import time
from nets.yolo import YoloBody
from utils.utils import (cvtColor, get_classes, preprocess_input,
                         resize_image, show_config)
from utils.utils_bbox import DecodeBox

class YOLO(object):
    _defaults = {
        #训练文件
        "model_path": 'logs/ep995-loss1.765-val_loss3.642.pth',
        "classes_path": 'model_data/voc_classes.txt',
        #     输入图片的大小，必须为 32 的倍数。
        "input_shape": [640, 640],
        #     所使用到的 yolov8 的版本：
        "phi": 's',
        "confidence": 0.5,
        "nms_iou": 0.3,
        "letterbox_image": True,
        #   用 Cuda
        "cuda": True,}
    @classmethod
    def get_defaults(cls, n):
        if n in cls._defaults:
            return cls._defaults[n]
        else:
            return "Unrecognized attribute name '" + n + "'"
    #     初始化 YOLO
    def __init__(self, **kwargs):
        self.__dict__.update(self._defaults)
        for name, value in kwargs.items():
            setattr(self, name, value)
            self._defaults[name] = value
        self.class_names, self.num_classes = get_classes(self.classes_path)
        self.bbox_util = DecodeBox(self.num_classes, (self.input_shape[0], self.input_shape[1]))
        # 设置不同的颜色画框
        hsv_tuples = [(x / self.num_classes, 1., 1.) for x in range(self.num_classes)]
        self.colors = list(map(lambda x: colorsys.hsv_to_rgb(*x), hsv_tuples))
        self.colors = list(map(lambda x: (int(x[0] * 255), int(x[1] * 255), int(x[2] * 255)), self.colors))
        self.generate()
        show_config(**self._defaults)
```

```
    #   生成模型
    def generate(self, onnx=False):
        #   建立 yolo 模型，载入 yolo 模型的权重
        self.net = YoloBody(self.input_shape, self.num_classes, self.phi)
        device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
        self.net.load_state_dict(torch.load(self.model_path, map_location=device))
        self.net = self.net.fuse().eval()
        print('{} model, and classes loaded.'.format(self.model_path))
        if not onnx:
            if self.cuda:
                self.net = nn.DataParallel(self.net)
                self.net = self.net.cuda()
    #   检测
    def detect_image(self, image, crop=False, count=False):
        image_shape = np.array(np.shape(image)[0:2])
        image = cvtColor(image)
       image_data=resize_image(image,(self.input_shape[1],self.input_shape[0]),self.letterbox_image)
image_data=np.expand_dims(np.transpose(preprocess_input(np.array(image_data,dtype='float32')), (2, 0, 1)), 0)
        with torch.no_grad():
            images = torch.from_numpy(image_data)
            if self.cuda:
                images = images.cuda()
            outputs = self.net(images)
            outputs = self.bbox_util.decode_box(outputs)
            #   将预测框进行堆叠
            results = self.bbox_util.non_max_suppression(outputs, self.num_classes, self.input_shape,
image_shape,self.letterbox_image,conf_thres=self.confidence,nms_thres=self.nms_iou)
            if results[0] is None:
                return image
            top_label = np.array(results[0][:, 5], dtype='int32')
            top_conf = results[0][:, 4]
            top_boxes = results[0][:, :4]
        #   设置字体与边框厚度
        font = ImageFont.truetype(font='model_data/simhei.ttf', size=np.floor(5e-2 * image.size[1] + 0).astype('int32'))
        thickness = int(max((image.size[0] + image.size[1]) // np.mean(self.input_shape), 2))
        #   计数
        if count:
            print("top_label:", top_label)
            classes_nums = np.zeros([self.num_classes])
            for i in range(self.num_classes):
                num = np.sum(top_label == i)
                if num > 0:
                    print(self.class_names[i], " : ", num)
                classes_nums[i] = num
```

```
            print("classes_nums:", classes_nums)
        #   目标裁剪
        if crop:
            for i, c in list(enumerate(top_boxes)):
                top, left, bottom, right = top_boxes[i]
                top = max(0, np.floor(top).astype('int32'))
                left = max(0, np.floor(left).astype('int32'))
                bottom = min(image.size[1], np.floor(bottom).astype('int32'))
                right = min(image.size[0], np.floor(right).astype('int32'))
                dir_save_path = "img_crop"
                if not os.path.exists(dir_save_path):
                    os.makedirs(dir_save_path)
                crop_image = image.crop([left, top, right, bottom])
                crop_image.save(os.path.join(dir_save_path, "crop_" + str(i) + ".png"), quality=95, subsampling=0)
                print("save crop_" + str(i) + ".png to " + dir_save_path)
        #    图像绘制
        for i, c in list(enumerate(top_label)):
            predicted_class = self.class_names[int(c)]
            box = top_boxes[i]
            score = top_conf[i]
            top, left, bottom, right = box
            top = max(0, np.floor(top).astype('int32'))
            left = max(0, np.floor(left).astype('int32'))
            bottom = min(image.size[1], np.floor(bottom).astype('int32'))
            right = min(image.size[0], np.floor(right).astype('int32'))
            label = '{} {:.2f}'.format(predicted_class, score)
            draw = ImageDraw.Draw(image)
            label_size = draw.textsize(label, font)
            label = label.encode('utf-8')
            print(label, top, left, bottom, right)
            if top - label_size[1] >= 0:
                text_origin = np.array([left, top - label_size[1]])
            else:
                text_origin = np.array([left, top + 1])
            for i in range(thickness):
                draw.rectangle([left + i, top + i, right - i, bottom - i], outline=self.colors[c])
            draw.rectangle([tuple(text_origin), tuple(text_origin + label_size)], fill=self.colors[c])
            draw.text(text_origin, str(label, 'UTF-8'), fill=(0, 0, 0), font=font)
            del draw
        return image
#获取热力图结果
    def detect_heatmap(self, image, heatmap_save_path):
        import cv2
        import matplotlib.pyplot as plt
        def sigmoid(x):
```

```
                y = 1.0 / (1.0 + np.exp(-x))
                return y
            image = cvtColor(image)
            image_data = resize_image(image, (self.input_shape[1], self.input_shape[0]), self.letterbox_image)
            image_data = np.expand_dims(np.transpose(preprocess_input(np.array(image_data, dtype='float32')), (2, 0, 1)),
0)
            with torch.no_grad():
                images = torch.from_numpy(image_data)
                if self.cuda:
                    images = images.cuda()
                dbox, cls, x, anchors, strides = self.net(images)
                outputs = [xi.split((xi.size()[1] - self.num_classes, self.num_classes), 1)[1] for xi in x]
            plt.imshow(image, alpha=1)
            plt.axis('off')
            mask = np.zeros((image.size[1], image.size[0]))
            for sub_output in outputs:
                sub_output = sub_output.cpu().numpy()
                b, c, h, w = np.shape(sub_output)
                sub_output = np.transpose(np.reshape(sub_output, [b, -1, h, w]), [0, 2, 3, 1])[0]
                score = np.max(sigmoid(sub_output[..., :]), -1)
                score = cv2.resize(score, (image.size[0], image.size[1]))
                normed_score = (score * 255).astype('uint8')
                mask = np.maximum(mask, normed_score)
            plt.imshow(mask, alpha=0.5, interpolation='nearest', cmap="jet")
            plt.axis('off')
            plt.subplots_adjust(top=1, bottom=0, right=1, left=0, hspace=0, wspace=0)
            plt.margins(0, 0)
            plt.savefig(heatmap_save_path, dpi=200, bbox_inches='tight', pad_inches=-0.1)
            print("Save to the " + heatmap_save_path)
            plt.show()
#获取输出的 map_out 结果
        def get_map_txt(self, image_id, image, class_names, map_out_path):
            f = open(os.path.join(map_out_path, "detection-results/" + image_id + ".txt"), "w", encoding='utf-8')
            image_shape = np.array(np.shape(image)[0:2])
            image = cvtColor(image)
           image_data=resize_image(image,(self.input_shape[1],self.input_shape[0]),self.letterbox_image)
            image_data=np.expand_dims(np.transpose(preprocess_input(np.array(image_data, dtype='float32')), (2, 0, 1)), 0)
            with torch.no_grad():
                images = torch.from_numpy(image_data)
                if self.cuda:
                    images = images.cuda()
                outputs = self.net(images)
                outputs = self.bbox_util.decode_box(outputs)
                results = self.bbox_util.non_max_suppression(outputs, self.num_classes, self.input_shape,
image_shape, self.letterbox_image, conf_thres=self.confidence,nms_thres=self.nms_iou)
```

```
            if results[0] is None:
                return
            top_label = np.array(results[0][:, 5], dtype='int32')
            top_conf = results[0][:, 4]
            top_boxes = results[0][:, :4]
        for i, c in list(enumerate(top_label)):
            predicted_class = self.class_names[int(c)]
            box = top_boxes[i]
            score = str(top_conf[i])
            top, left, bottom, right = box
            if predicted_class not in class_names:
                continue
            f.write("%s %s %s %s %s %s\n" % (
            predicted_class, score[:6], str(int(left)), str(int(top)), str(int(right)), str(int(bottom))))
        f.close()
        return
```

程序编号	4	程序说明	voc_annotation.py（验证集和测试集划分）	编程软件	Python

```
import os
import random
import xml.etree.ElementTree as ET
import numpy as np
from utils.utils import get_classes
annotation_mode         = 0
classes_path            = 'model_data/voc_classes.txt'
trainval_percent        = 0.9
train_percent           = 0.9
#     默认指向数据集
VOCdevkit_path    = 'VOCdevkit'
VOCdevkit_sets    = [('2007', 'train'), ('2007', 'val')]
classes, _          = get_classes(classes_path)
#     统计目标数量
photo_nums    = np.zeros(len(VOCdevkit_sets))
nums            = np.zeros(len(classes))
def convert_annotation(year, image_id, list_file):
    in_file = open(os.path.join(VOCdevkit_path, 'VOC%s/Annotations/%s.xml'%(year, image_id)), encoding='utf-8')
    tree=ET.parse(in_file)
    root = tree.getroot()

    for obj in root.iter('object'):
        difficult = 0
        if obj.find('difficult')!=None:
            difficult = obj.find('difficult').text
```

```
            cls = obj.find('name').text
            if cls not in classes or int(difficult)==1:
                continue
            cls_id = classes.index(cls)
            xmlbox = obj.find('bndbox')
            b = (int(float(xmlbox.find('xmin').text)), int(float(xmlbox.find('ymin').text)), int(float(xmlbox.find('xmax').text)),
int(float(xmlbox.find('ymax').text)))
            list_file.write(" " + ",".join([str(a) for a in b]) + ',' + str(cls_id))

            nums[classes.index(cls)] = nums[classes.index(cls)] + 1

if __name__ == "__main__":
    random.seed(0)
    if " " in os.path.abspath(VOCdevkit_path):
        raise ValueError("path ERROR")
    if annotation_mode == 0 or annotation_mode == 1:
        print("Generate txt in ImageSets.")
        xmlfilepath     = os.path.join(VOCdevkit_path, 'VOC2007/Annotations')
        saveBasePath    = os.path.join(VOCdevkit_path, 'VOC2007/ImageSets/Main')
        temp_xml        = os.listdir(xmlfilepath)
        total_xml       = []
        for xml in temp_xml:
            if xml.endswith(".xml"):
                total_xml.append(xml)
        num     = len(total_xml)
        list    = range(num)
        tv      = int(num*trainval_percent)
        tr      = int(tv*train_percent)
        trainval= random.sample(list,tv)
        train   = random.sample(trainval,tr)

        print("train and val size",tv)
        print("train size",tr)
        ftrainval   = open(os.path.join(saveBasePath,'trainval.txt'), 'w')
        ftest       = open(os.path.join(saveBasePath,'test.txt'), 'w')
        ftrain      = open(os.path.join(saveBasePath,'train.txt'), 'w')
        fval        = open(os.path.join(saveBasePath,'val.txt'), 'w')

        for i in list:
            name=total_xml[i][:-4]+'\n'
            if i in trainval:
                ftrainval.write(name)
                if i in train:
                    ftrain.write(name)
                else:
```

```
                    fval.write(name)
            else:
                ftest.write(name)
        ftrainval.close()
        ftrain.close()
        fval.close()
        ftest.close()
        print("Generate txt in ImageSets done.")

    if annotation_mode == 0 or annotation_mode == 2:
        print("Generate 2007_train.txt and 2007_val.txt for train.")
        type_index = 0
        for year, image_set in VOCdevkit_sets:
            image_ids = open(os.path.join(VOCdevkit_path, 'VOC%s/ImageSets/Main/%s.txt'%(year, image_set)), encoding='utf-8').read().strip().split()
            list_file = open('%s_%s.txt'%(year, image_set), 'w', encoding='utf-8')
            for image_id in image_ids:
                list_file.write('%s/VOC%s/JPEGImages/%s.jpg'%(os.path.abspath(VOCdevkit_path), year, image_id))

                convert_annotation(year, image_id, list_file)
                list_file.write('\n')
            photo_nums[type_index] = len(image_ids)
            type_index += 1
            list_file.close()
        print("Generate 2007_train.txt and 2007_val.txt for train done.")

        def printTable(List1, List2):
            for i in range(len(List1[0])):
                print("|", end=' ')
                for j in range(len(List1)):
                    print(List1[j][i].rjust(int(List2[j])), end=' ')
                    print("|", end=' ')
                print()

        str_nums = [str(int(x)) for x in nums]
        tableData = [
            classes, str_nums
        ]
        colWidths = [0]*len(tableData)
        len1 = 0
        for i in range(len(tableData)):
            for j in range(len(tableData[i])):
                if len(tableData[i][j]) > colWidths[i]:
                    colWidths[i] = len(tableData[i][j])
        printTable(tableData, colWidths)
```

```
        if photo_nums[0] <= 500:
            print("ERROR NULL")

        if np.sum(nums) == 0:
            print("NULL")
```

程序编号	4	程序说明	voc_annotation.py（验证集和测试集划分）	编程软件	Python

```
import os
import random
import xml.etree.ElementTree as ET
import numpy as np
from utils.utils import get_classes
annotation_mode         = 0
classes_path            = 'model_data/voc_classes.txt'
trainval_percent        = 0.9
train_percent           = 0.9
#   默认指向数据集
VOCdevkit_path  = 'VOCdevkit'
VOCdevkit_sets  = [('2007', 'train'), ('2007', 'val')]
classes, _          = get_classes(classes_path)
#   统计目标数量
photo_nums  = np.zeros(len(VOCdevkit_sets))
nums            = np.zeros(len(classes))
def convert_annotation(year, image_id, list_file):
    in_file = open(os.path.join(VOCdevkit_path, 'VOC%s/Annotations/%s.xml'%(year, image_id)), encoding='utf-8')
    tree=ET.parse(in_file)
    root = tree.getroot()

    for obj in root.iter('object'):
        difficult = 0
        if obj.find('difficult')!=None:
            difficult = obj.find('difficult').text
        cls = obj.find('name').text
        if cls not in classes or int(difficult)==1:
            continue
        cls_id = classes.index(cls)
        xmlbox = obj.find('bndbox')
        b = (int(float(xmlbox.find('xmin').text)), int(float(xmlbox.find('ymin').text)), int(float(xmlbox.find('xmax').text)),
int(float(xmlbox.find('ymax').text)))
        list_file.write(" " + ",".join([str(a) for a in b]) + ',' + str(cls_id))

        nums[classes.index(cls)] = nums[classes.index(cls)] + 1
```

```
if __name__ == "__main__":
    random.seed(0)
    if " " in os.path.abspath(VOCdevkit_path):
        raise ValueError("path ERROR")
    if annotation_mode == 0 or annotation_mode == 1:
        print("Generate txt in ImageSets.")
        xmlfilepath         = os.path.join(VOCdevkit_path, 'VOC2007/Annotations')
        saveBasePath        = os.path.join(VOCdevkit_path, 'VOC2007/ImageSets/Main')
        temp_xml              = os.listdir(xmlfilepath)
        total_xml             = []
        for xml in temp_xml:
            if xml.endswith(".xml"):
                total_xml.append(xml)
        num         = len(total_xml)
        list        = range(num)
        tv          = int(num*trainval_percent)
        tr          = int(tv*train_percent)
        trainval= random.sample(list,tv)
        train     = random.sample(trainval,tr)

        print("train and val size",tv)
        print("train size",tr)
        ftrainval     = open(os.path.join(saveBasePath,'trainval.txt'), 'w')
        ftest           = open(os.path.join(saveBasePath,'test.txt'), 'w')
        ftrain          = open(os.path.join(saveBasePath,'train.txt'), 'w')
        fval            = open(os.path.join(saveBasePath,'val.txt'), 'w')

        for i in list:
            name=total_xml[i][:-4]+'\n'
            if i in trainval:
                ftrainval.write(name)
                if i in train:
                    ftrain.write(name)
                else:
                    fval.write(name)
            else:
                ftest.write(name)
        ftrainval.close()
        ftrain.close()
        fval.close()
        ftest.close()
        print("Generate txt in ImageSets done.")

    if annotation_mode == 0 or annotation_mode == 2:
```

```
        print("Generate 2007_train.txt and 2007_val.txt for train.")
        type_index = 0
        for year, image_set in VOCdevkit_sets:
            image_ids = open(os.path.join(VOCdevkit_path, 'VOC%s/ImageSets/Main/%s.txt'%(year, image_set)),
encoding='utf-8').read().strip().split()
            list_file = open('%s_%s.txt'%(year, image_set), 'w', encoding='utf-8')
            for image_id in image_ids:
                list_file.write('%s/VOC%s/JPEGImages/%s.jpg'%(os.path.abspath(VOCdevkit_path), year, image_id))

                convert_annotation(year, image_id, list_file)
                list_file.write('\n')
            photo_nums[type_index] = len(image_ids)
            type_index += 1
            list_file.close()
        print("Generate 2007_train.txt and 2007_val.txt for train done.")

        def printTable(List1, List2):
            for i in range(len(List1[0])):
                print("|", end=' ')
                for j in range(len(List1)):
                    print(List1[j][i].rjust(int(List2[j])), end=' ')
                    print("|", end=' ')
                print()

        str_nums = [str(int(x)) for x in nums]
        tableData = [
            classes, str_nums
        ]
        colWidths = [0]*len(tableData)
        len1 = 0
        for i in range(len(tableData)):
            for j in range(len(tableData[i])):
                if len(tableData[i][j]) > colWidths[i]:
                    colWidths[i] = len(tableData[i][j])
        printTable(tableData, colWidths)

        if photo_nums[0] <= 500:
            print("ERROR NULL")

        if np.sum(nums) == 0:
            print("NULL")
```

程序编号	5	程序说明	plot_figure（作图）	编程软件	Matlab

```
N = 10; %  每条数据个数
Y = [1,1.5,2,2.5,3];%
Z1=[79.60	81.28	78.29	80.16	77.19	81.91	79.36	80.46	79.70	76.33	72.30	86.61
88.64	89.67	88.04	89.21	87.10	89.44	88.83	89.40	89.14	89.61	89.37	90.45
90.81	91.33	89.97	90.42	90.91	88.50	90.66	91.22	90.31	82.75	82.40	93.30
75.69	77.15	73.16	76.65	83.59	80.41	70.17	80.97	80.62	69.11	59.63	74.80
85.27	86.93	84.26	86.70	86.61	87.08	87.04	92.60	91.88	88.36	85.37	97.60
]
figure('color','w','position',[0,0,800,600])
ax = newplot();hold on;
FancyBar3(X,Y,Z1);
ax.YTick = Y;
ax.XTick = 1:N;
ax.ZTick = 0:3;
ax.FontSize = 14;
%  这里对应 Y 轴 3 个 ticks
ax.YTickLabel={'mAP','Fsc','Pre','Rec','Acc'};
xlabel('model','fontsize',14,'FontName','TimesNewRoman','FontSize',18);
ylabel('group','fontsize',14,'FontName','TimesNewRoman','FontSize',18);
zlabel('score/%','fontsize',14,'FontName','TimesNewRoman');
yyy = get(ax,'YLim');
ax.XGrid = 'off';
ax.YGrid = 'off';
ax.ZGrid = 'on';
grid on
iou=[68     272   394   763   1958 637]
ax = newplot();hold on;
figure(4)
bar(iou)
ax.XTickLabel={'0','<0.5','0.5-0.6','0.6-0.7','0.7-0.8','0.8-0.9','>0.9'};
xlabel('精度')
ylabel('数量')

function [ax] = FancyBar3(varargin)
%% Output
% ax                : Handle to the coordinate axes used for drawing
%===========================================================================%
% read & check parameters
[X,Y,Z,B3,ax] = inputParamCheck(varargin,nargin);
B3 = StructCompletion(B3,X);
```

```
TempFaceAlpha    = B3.TempFaceAlpha;
colorarr = B3.colorarr;
LineWidth = B3.LineWidth;
% draw plane by plane
for looptimeI = 1:length(Y)
    if looptimeI==length(Y)
        TempYWidth = B3.TempYWidth*(Y(looptimeI)-Y(looptimeI-1));
    else
        TempYWidth = B3.TempYWidth*(Y(looptimeI+1)-Y(looptimeI));
    end
    Xdata = X(looptimeI,:);
    Zdata = Z(looptimeI,:);
    Ydata = Y(looptimeI);
    TempWidth = (X(looptimeI,2)-X(looptimeI,1))*B3.dWidth;
    for looptimeJ = 1:length(Xdata)
        TempXdata = Xdata(looptimeJ);
        TempZdata = Zdata(looptimeJ);
        fillres = fill3([TempXdata-TempWidth/2,TempXdata-TempWidth/2,...
            TempXdata+TempWidth/2,TempXdata+TempWidth/2],...
            [Ydata-TempYWidth/2,Ydata-TempYWidth/2,Ydata-TempYWidth/2,Ydata-TempYWidth/2],...
            [0,TempZdata,TempZdata,0],colorarr(looptimeI,:));
        fillres.FaceAlpha = TempFaceAlpha;
        fillres.LineWidth = LineWidth;
        fillres = fill3([TempXdata-TempWidth/2,TempXdata-TempWidth/2,...
            TempXdata+TempWidth/2,TempXdata+TempWidth/2],...
            [Ydata+TempYWidth/2,Ydata+TempYWidth/2,Ydata+TempYWidth/2,Ydata+TempYWidth/2],...
            [0,TempZdata,TempZdata,0],colorarr(looptimeI,:));
        fillres.FaceAlpha = TempFaceAlpha;
        fillres.LineWidth = B3.LineWidth;
        fillres = fill3([TempXdata-TempWidth/2,TempXdata-TempWidth/2,...
            TempXdata-TempWidth/2,TempXdata-TempWidth/2],...
            [Ydata+TempYWidth/2,Ydata+TempYWidth/2,Ydata-TempYWidth/2,Ydata-TempYWidth/2],...
            [0,TempZdata,TempZdata,0],colorarr(looptimeI,:));
        fillres.FaceAlpha = TempFaceAlpha;
        fillres.LineWidth = B3.LineWidth;
        fillres = fill3([TempXdata+TempWidth/2,TempXdata+TempWidth/2,...
            TempXdata+TempWidth/2,TempXdata+TempWidth/2],...
            [Ydata+TempYWidth/2,Ydata+TempYWidth/2,Ydata-TempYWidth/2,Ydata-TempYWidth/2],...
            [0,TempZdata,TempZdata,0],colorarr(looptimeI,:));
        fillres.FaceAlpha = TempFaceAlpha;
        fillres.LineWidth = B3.LineWidth;
        fillres = fill3([TempXdata-TempWidth/2,TempXdata+TempWidth/2,...
            TempXdata+TempWidth/2,TempXdata-TempWidth/2],...
            [Ydata-TempYWidth/2,Ydata-TempYWidth/2,Ydata+TempYWidth/2,Ydata+TempYWidth/2],...
            [TempZdata,TempZdata,TempZdata,TempZdata],colorarr(looptimeI,:));
```

```
            fillres.FaceAlpha = TempFaceAlpha;
            fillres.LineWidth = B3.LineWidth;
            hold on;
        end
end
view(B3.ViewAngle);
xlim(ax,[min(min(X)),max(max(X))]);
grid on;
deltaX = (max(max(X))-min(min(X)))*0.05;
xlim([min(min(X))-TempWidth/2-deltaX,max(max(X))+TempWidth/2+deltaX]);
deltaY = (max(max(Y))-min(min(Y)))*0.05;
ylim([min(min(Y))-TempYWidth/2-deltaY,max(max(Y))+TempYWidth/2+deltaY]);
end

function [X,Y,Z,B3settings,ax] = inputParamCheck(V,N)
if N==5
    X = V{1};
    Y = V{2};
    Z = V{3};
    ax = V{4};
    B3settings = V{5};
elseif N==4
    X = V{1};
    Y = V{2};
    Z = V{3};
    ax = V{4};
    B3settings = [];
elseif N==3
    X = V{1};
    Y = V{2};
    Z = V{3};
    ax = gca;
    B3settings = [];
else
    error('Check the number of input parameters')
end
end
function B3C = StructCompletion(B3,X)
if ~isstruct(B3)
    B3C.dWidth          = 0.4;
    B3C.LineWidth       = 0.7;
    B3C.TempYWidth      = 0.3;
    B3C.TempFaceAlpha   = 0.7;
    B3C.ViewAngle       = [45,45];
    B3C.colorarr        = [1 0.98039 0.94118
```

```
                                1 0.89412 0.7098
                                0.67843 0.84706 0.90196
                                0.3884 0.72157 1
                                0 0 1
                                   ];
else
    B3C = B3;
    if ~isfield(B3C,'dWidth')
        B3C.dWidth = 0.4;
    end
    if ~isfield(B3C,'LineWidth')
        B3C.LineWidth = 0.7;
    end
    if ~isfield(B3C,'TempYWidth')
        B3C.TempYWidth = 0.3;
    end
    if ~isfield(B3C,'TempFaceAlpha')
        B3C.TempFaceAlpha = 0.7;
    end
    if ~isfield(B3C,'colorarr')
        B3C.colorarr = winter(size(X,1));
    end
    if ~isfield(B3C,'ViewAngle')
        B3C.ViewAngle = [45,45];
    end
end
end
```